

Software-as-a-Graph: Dependency-Graph Analysis and Learning for Pre-Deployment Cascading-Failure Ranking in Publish-Subscribe Systems

Ibrahim Onuralp Yigit^{a,*}, Feza Buzluca^a

^a*Department of Computer Engineering, Istanbul Technical University, Istanbul, 34469, Turkey*

Abstract

Publish-subscribe middleware decouples components in space and time, and in doing so hides the paths along which failures cascade. Architects need to know which components are systemically critical before deployment, when no runtime telemetry exists. We present Software-as-a-Graph (SaG), a framework that turns architecture models into typed multigraphs and, through six publish-subscribe rules, derives an explicit graph of logical dependencies. It ranks components by cascading-failure impact with closed-form, learned and hybrid engines, and explains each flagged component with an ISO/IEC 25010 reliability and maintainability profile. Under leave-one-scenario-out cross-validation over twelve synthetic architectures, the derived dependency graph is the decisive ingredient. Counting a component's dependents on it ranks simulated cascade impact at Spearman $\rho = 0.764$, above closed-form centrality on all twelve held-out architectures and far above counting raw connections (+0.565). On hand-authored models of five open-source systems, counting transitive dependents reaches $\rho = 0.938$ without any training. Graph neural networks gain +0.08 to +0.11 when they read the dependency graph instead of the raw multigraph, reaching $\rho = 0.748$: level with dependency counting, not above it. Hybrid engines, in which a learned model corrects closed-form centrality, beat that centrality on 11 of 12 architectures under decision rules registered before their runs. Declared Quality-of-Service contracts help learned engines through three node-level coupling features. Dependency counting takes milliseconds. Deriving explicit dependency graphs from architecture models thus makes cascading-failure risk rankable before deployment.

Keywords: Dependency graphs, cascading failures, publish-subscribe, graph neural networks, software architecture, dependability, empirical study

1. Introduction

1.1. Motivation and Problem

Large distributed systems increasingly communicate through asynchronous publish-subscribe (pub-sub) middleware: ROS 2 in autonomous driving [1], Apache Kafka in enterprise event streams [2], DDS in cyber-physical systems [3], MQTT in IoT [4], and cloud-native microservices [5, 6]. Pub-sub decouples producers and consumers in space, time and synchronization [7]. Components interact through topics and brokers rather than direct references, and deployment-time Quality-of-Service (QoS) policies govern reliability, durability, priority and deadlines.

The same decoupling hides how failures spread. Publishers and subscribers share no direct link, so outages, head-of-line blocking and backpressure propagate along concealed paths through brokers, shared topics, colocated hosts and shared libraries [8, 9]. These failures take two forms. In *sequential cascades*, a slow subscriber fills a broker queue and gradually starves its publishers [10]. In *simultaneous blasts*, a shared-library crash or host outage takes down every colocated service at once. Neither architecture diagrams

*Corresponding author

Email addresses: yigiti@itu.edu.tr (Ibrahim Onuralp Yigit), buzluca@itu.edu.tr (Feza Buzluca)

nor static call graphs show these mechanisms. The cheapest time to reduce the risk is before deployment, at design and continuous-integration time [11, 12], when no runtime telemetry exists. Architects therefore need to know, from configuration manifests alone, which components, topics and links are systemically critical and why.

Existing practice leaves this gap open, which we call the **Architecture–Code Gap**: a system can have bug-free code in every service and still be fragile through hidden single points of failure or mismatched QoS contracts [13, 14]. Architecture evaluations such as ATAM rely on manual elicitation [15]. Static code analysis inspects services in isolation [16, 17]. Chaos engineering needs a provisioned cluster [18]. Homogeneous centrality flattens typed topologies into untyped graphs [19, 20]. Learned models could combine these structural cues, but on their own they produce risk scores without actionable explanations. What is missing is a representation that makes pub-sub failure paths explicit, and evidence on which analyzer to trust on it. This paper shows that once those paths are derived, they can be counted: a component’s dependents on the derived graph rank its cascade impact well, before any code is deployed.

1.2. The Software-as-a-Graph (SaG) Approach

Software-as-a-Graph (SaG) is a pre-deployment static system analysis framework for event-driven architectures (Figure 1). It (1) models an architecture as a typed, directed multigraph over Applications, Brokers, Topics, Execution Hosts and Shared Libraries (Section 3.1); (2) derives from it, through six publish–subscribe rules, an explicit **DEPENDS_ON** dependency graph that captures both sequential cascades and simultaneous blasts (Section 3.2); (3) ranks components by predicted cascade impact with training-free engines on that graph, graph-learning engines, and hybrid engines in which a learned model corrects a closed-form score (Section 4); and (4) explains flagged components with an ISO/IEC 25010 Reliability–Maintainability (RM) profile (Section 5). Predictors read only the analysis graph. Ground-truth impact comes from independent simulators that run on the raw structural topology (Section 4.4). The central thesis is that the representation, not the choice of analyzer, is the decisive investment. The evidence supports it at every level. On the derived dependency graph, simply counting a component’s dependents ranks its simulated cascade impact at $\rho = 0.764$ on held-out architectures, where counting a component’s raw connections reaches only 0.199, and counting transitive dependents reaches 0.938 on models of real systems. Closed-form centrality improves on every held-out architecture when it is computed on the Application–Library dependency graph. Graph neural networks gain +0.08 to +0.11 when they read it instead of the raw multigraph.

1.3. Research Questions

- **RQ1 (Ranking accuracy):** *How accurately do SaG’s closed-form, learned and hybrid engines rank components by cascading-failure impact on unseen architectures, compared with structural baselines and with dependency counts on the derived graph?*
- **RQ2 (What learning needs):** *What do learned engines need: which graph they read, relation-specific (typed) parameters, or QoS inputs, once model capacity and edge-channel width are matched?*
- **RQ3 (Transfer):** *How well do engines trained on synthetic architectures transfer zero-shot to independently authored models of five open-source systems?*
- **RQ4 (Cost):** *What does the analysis cost at CI/CD time, which stage dominates, and how does it compare with running the simulation directly?*

The primary contrast, the matched control, both hybrid engines, the dependency counts, the dependency-graph learners and the derivation check were each registered with a decision rule before their results existed. The last three (Amendments 7, 9 and 10) are exploratory families outside the confirmatory omnibus. Supplementary Section S24 logs every change as an amendment and maps these four questions onto the five of the registered analysis plan.

1.4. Contributions

Evaluated under leave-one-scenario-out (LOSO) cross-validation over twelve synthetic architectures and zero-shot on five open-source system models, this paper contributes:

1. **A publish–subscribe dependency derivation that makes failure paths countable** (Section 3). Six rules turn topics, brokers, hosts and shared libraries into explicit logical dependencies that separate sequential cascades from simultaneous blasts. On the derived graph, counting a component’s dependents ranks its cascade impact at $\rho = 0.764$, above closed-form centrality on all twelve held-out architectures (+0.211) and far above counting raw connections (+0.565, 12/12 folds). Counting transitive dependents transfers without training to five open-source system models at $\rho = 0.938$. On held-out architectures the derived library rule adds +0.058 to transitive reach.
2. **A portfolio of closed-form, learned and hybrid engines, and the finding that learning pays off when it reads the dependency graph** (Section 4). Centrality on the Application–Library dependency graph lifts closed-form ranking from 0.349 to 0.553 on every held-out architecture, and reading that graph lifts the same graph neural networks by +0.08 to +0.11 over reading the raw multigraph, to $\rho = 0.748$, level with dependency counting. Hybrid engines that learn a correction to closed-form centrality beat it on 11 of 12 folds (+0.103 and +0.130), under decision rules registered before their runs, and remain significant under a Holm correction pooled over all thirteen confirmatory contrasts.
3. **Evidence on what graph learning needs and how far it transfers** (Section 7). Learned engines need messages from a component’s dependents and declared coupling features; relation-specific weights add nothing once capacity is matched. The QoS gain (+0.073 on 10 of 12 folds) is carried by three declared-coupling node features rather than the 16-D edge encoding, so an untyped attention network suffices. Trained only on synthetic data, learned engines transfer zero-shot to five open-source system models at $\rho = 0.760\text{--}0.831$, far above closed-form centrality (0.511–0.526). A map of where graph learning helps and where it fails completes this evidence (Section 8.2).
4. **A standards-grounded explanation layer** (Section 5) that attributes each flagged component to ISO/IEC 25010 Availability, Fault Tolerance or Maintainability, and so names the remediation it calls for: replication, circuit breakers or decoupling.
5. **A reproducible benchmark and cost profile**: seventeen architectures totaling 2,812 components, twelve of which regenerate byte-identically from committed configurations. Every reported table value is mechanically reconciled against released artifacts. Dependency counting takes milliseconds, and neural inference 56 ms on a 2,000-component architecture (Section 7.4).

A previous conference paper [21] introduced the preliminary multigraph and deterministic quality model on synthetic topologies. This paper adds the dependency-count and learned engines on the derived graph, the hybrid engines, the QoS edge encoding, LOSO and zero-shot evaluation, the matched control, and the cost profile, and it repositions that quality model as the explanation layer.

Section 2 reviews related work, Sections 3–5 present the model, engines and explanation layer, Sections 6–7 the evaluation, Section 8 the discussion and threats, and Section 9 concludes.

2. Related Work

2.1. Dependability Analysis of Distributed Systems

Runtime approaches to dependability, such as broker clustering, backpressure, autoscaling, failover and chaos engineering [18], require a running cluster, can disrupt service, and consume substantial compute. That compute is itself a concern of green software engineering [22, 23, 24]. Architecture-based reliability prediction has a long history. Cheung’s absorbing Markov chain [25] and the state-, path- and additive models surveyed by Goseva-Popstojanova and Trivedi [26] and Immonen and Niemelä [27] are examples, as are the Palladio Component Model [28], layered queueing networks [29] and the AADL Error Model Annex [30]. These approaches answer broader questions but need operational profiles and failure rates that are unavailable at commit time. SaG asks a narrower question from manifests alone: whose failure reaches furthest in the declared topology?

Telemetry-driven methods diagnose faults in running microservices. Examples are Seer [31] and Sage [32], MicroRCA [33], TraceRCA [34], and GNN-based DeepTraLog [35], Eadro [36] and MicroCause [37] (reviewed in [38]). In synchronous microservices, cascades stem from thread-pool exhaustion, RPC timeouts and retry

storms [39]. In pub-sub systems they spread through queue saturation and message starvation. All of these methods need traces or metrics from a live system; SaG works before any code runs. SaG’s question is also that of change impact analysis [40] and of dependency management in microservice fleets [41]. Ranking by impact relates to learning-to-rank defect prediction [42], which optimizes the ranking measure directly, as our listwise loss does (Section 4.2).

2.2. Static Code Analysis and Static System Analysis

Static code analysis (SCA) tools such as SonarQube [16] measure complexity [43], cohesion and coupling [17, 44] within individual services to flag defect-prone modules [45, 46, 47, 48]. SCA cannot see inter-service messaging, broker saturation or cross-host propagation. Architecture recovery tools reconstruct system-level structure from code [49, 50], and HAROS [51] checks ROS systems statically before launch. SaG’s static system analysis (SSA) uses the declared topology instead, propagating code-level metrics across architectural dependencies. Architecture-level coupling metrics count a service’s consumers, as afferent coupling [52], the Absolute Importance of the Service [53] and service fan-in [54]. In publish–subscribe systems these counts are not directly observable, because consumers reach producers only through topics, brokers and shared libraries; SaG’s dependency derivation makes them computable, and Section 7.1 shows that they then rank cascade impact well. This lets teams find structural anti-patterns [55, 56] and architectural technical debt [57] in CI/CD [58, 59].

2.3. Quality Models and Multi-Criteria Evaluation

ISO/IEC 25010:2023 [60] and ISO/IEC 25019:2023 [61] define product quality and quality in use. SaG covers the characteristics derivable from deployment topology, namely Availability, Fault Tolerance and the Maintainability sub-characteristics (Section 5.1), and links internal structural quality to external dependability [62, 63]. Aggregating metrics into an auditable score is a multi-criteria decision problem, for which the Analytic Hierarchy Process (AHP) [64] is standard. AHP’s consistency ratio detects inconsistent judgments but not matrices back-filled from a chosen answer, a distinction this study reports for its weights (Supplementary Section S4).

2.4. Graph Learning and Explainability

Centrality indices [19, 20, 65, 66] and cascade models of network robustness [10, 8, 9] assume homogeneous, usually undirected graphs. A single untyped score conflates structurally different elements, such as topics, libraries and hosts, and cannot say *why* a component is critical. Learned node-importance methods, including FINDER [67], DrBC [68] and PowerGraph [69], share the homogeneity assumption. Homogeneous GNNs (GCN [70], GraphSAGE [71], GAT [72]) discard relation identity unless it is supplied as a feature. Heterogeneous GNNs (RGCN [73], HAN [74], HGT [75], MAGNN [76]) learn relation-specific transformations. We evaluate both families under matched capacity (Section 7.2). Khodabandeh et al. [77] apply graph attention to microservice call graphs to predict future interactions. We instead predict the impact of removing a node from a declared topology. GNN explainers such as GNNExplainer [78] and PGExplainer [79] explain models in terms of their internal features. SaG’s explanation layer (Section 5) instead names ISO/IEC quality sub-characteristics and a remediation class for each flagged component.

3. The Software-as-a-Graph (SaG) Architectural Model

Figure 1 presents the end-to-end architecture of the SaG framework. The shared front end processes Architecture-as-Code manifests, constructs a typed multigraph, projects implicit runtime interactions throughout a QoS-weighted logical dependency layer, and extracts typed node properties. These features feed the ranking engines (Section 4) and, separately and with no shared parameters, the explanation layer (Section 5).

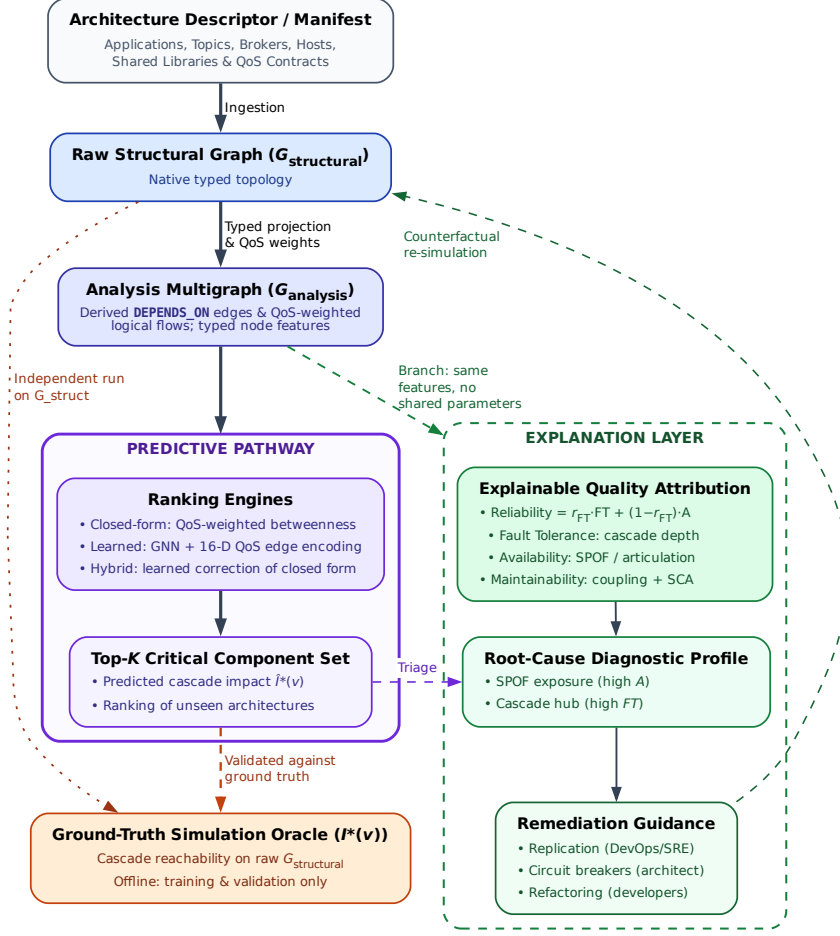


Figure 1: End-to-end architecture of the SaG framework. The predictive pathway runs down the centre: manifest ingestion, typed multigraph, QoS-weighted **DEPENDS_ON** projection with typed node properties, the ranking engines (closed-form, learned and hybrid; Figure 3), and the ranked critical set. The dashed edge marks the ground-truth simulation oracle, which operates only on $G_{\text{structural}}$, trains the predictor offline and takes no part in inference. The explanation layer re-enters from the analysis multigraph and shares no parameters with the predictor, reaching flagged components through triage rather than data flow.

3.1. Formal Multigraph Definition

A distributed system is described as a typed, weighted, directed multigraph:

$$\mathcal{G} = (V, E, \tau_V, \tau_E, w_V, w_E) \quad (1)$$

where:

- $V = V_{\text{app}} \cup V_{\text{broker}} \cup V_{\text{topic}} \cup V_{\text{host}} \cup V_{\text{lib}}$ holds the five entity types $\mathcal{T}_V$ of Table 1; V_{host} denotes physical or virtual *Execution Hosts*.
- E is the set of directed edges, and $\tau_V : V \rightarrow \mathcal{T}_V$ and $\tau_E : E \rightarrow \mathcal{T}_E$ assign entity and relation types.
- $w_V : V \rightarrow (0, 1]$ and $w_E : E \rightarrow (0, 1]$ weight entity criticality and connection strength. For Applications and Libraries, $w_V(v) = 1 - \text{CQP}(v)$, where CQP is a code-quality penalty computed from static code metrics (lines of code, cyclomatic complexity, coupling and cohesion); otherwise $w_V(v) = 1.0$.

3.2. Logical Dependency Derivation and QoS-Aware Weights

A link's strength depends on its QoS contract: a **RELIABLE** topic with **TRANSIENT_LOCAL** durability couples services more strongly than a **BEST_EFFORT** telemetry stream. Each topic t therefore carries a weight $w(t) \in (0, 1]$ combining its declared QoS with payload size and publication frequency:

Table 1: Entity types and structural edge types in the SaG model.

Entity Type ($\mathcal{T}_V$)	Architectural Role	Concrete System Examples
Application (V_{app})	Process producing/consuming messages	ROS 2 node, Kafka microservice, MQTT client
Broker (V_{broker})	Message routing and queuing intermediary	RabbitMQ exchange, Mosquitto, EMQX broker
Topic (V_{topic})	Named logical communication channel	<code>/sensor/lidar</code> , <code>orders.payment.completed</code>
Execution Host (V_{host})	Physical or virtualized execution environment	Bare-metal server, Kubernetes worker, Cloud VM
Library (V_{lib})	Shared software package or runtime dependency	Kafka client, OpenCV, Protobuf runtime
Structural Edge ($\mathcal{T}_E$)	Direction	Semantic Meaning
PUBLISHES_TO / SUBSCRIBES_TO	App/Library $\rightarrow$ Topic	Component publishes to / consumes from topic
ROUTES	Broker $\rightarrow$ Topic	Broker manages and routes topic traffic
RUNS_ON	App/Broker $\rightarrow$ Host	Process is hosted on host
CONNECTS_TO	Host $\rightarrow$ Host	Network link between hosts
USES	App $\rightarrow$ Library	Application links to shared library

Table 2: Notation used throughout. Entity and edge types: Table 1; simulation oracles: Section 4.3.

$G_{\text{structural}}$	Raw multigraph; oracles only	$Q(v)$	RM composite quality score
G_{analysis}	DEPENDS_ON projection; predictor input	ρ	Spearman ρ , full population
V_{app}	Application nodes; the scored population	$\rho_{>0}$	Spearman ρ , active stratum ($I^* > 0$)
$w(t), w(e)$	QoS topic weight, edge weight	Overlap@ K	Top- K set overlap, $K = 0.20 V_{\text{app}} $
$I^*(v)$	Primary cascade-reachability oracle	$I_{\text{dyn}}(v)$	Queue-flow oracle; convergent validity

$$w(t) = \alpha_{\text{top}} \cdot \text{QoS}(t) + \beta_{\text{top}} \cdot \text{SizeNorm}(t) + \gamma_{\text{top}} \cdot \text{FreqNorm}(t), \quad (\alpha_{\text{top}}, \beta_{\text{top}}, \gamma_{\text{top}}) = (0.75, 0.15, 0.10) \quad (2)$$

where the QoS term is an AHP-weighted aggregate of the declared contract:

$$\text{QoS}(t) = w_{\text{rel}} \cdot q_{\text{rel}} + w_{\text{dur}} \cdot q_{\text{dur}} + w_{\text{prio}} \cdot q_{\text{prio}}, \quad (w_{\text{rel}}, w_{\text{dur}}, w_{\text{prio}}) = (0.24, 0.62, 0.14) \quad (3)$$

Here $q_{\text{rel}} \in \{0, 1\}$ (best-effort, reliable), $q_{\text{dur}} \in \{0, 0.5, 1\}$ (volatile, transient-local, persistent) and $q_{\text{prio}} \in \{0, 0.5, 1\}$ (low, medium, high). Durability carries the highest weight because it determines whether state and message history survive restarts. The sub-weights come from a Saaty pairwise matrix with genuine second-eigenvalue spread and $CR = 0.016$ (Supplementary Section S4). Size and frequency are log-compressed and clamped:

$$\text{SizeNorm}(t) = \min \left(1.0, \frac{\log_2(1 + B(t))}{20} \right), \quad \text{FreqNorm}(t) = \min \left(1.0, \frac{\log_{10}(1 + F(t))}{3} \right) \quad (4)$$

where $B(t)$ is the payload in bytes (design envelope 1 MiB, the practical DDS sample limit) and $F(t)$ the publication frequency in Hz. $w(t)$ is clamped to $[0.01, 1]$ so that best-effort edges remain visible, and every PUBLISHES_TO, SUBSCRIBES_TO and ROUTES edge of t carries $w_E(e) = w(t)$ and the topic's QoS vector. The $(\alpha_{\text{top}}, \beta_{\text{top}}, \gamma_{\text{top}})$ split is a documented choice rather than a sensitive parameter: no point of its simplex changes any reported ordering (Supplementary Section S1).

3.2.1. Logical Dependency Projection (DEPENDS_ON)

Structural edges do not capture implicit runtime dependencies: a subscriber depends on a publisher, yet no edge joins them. SaG therefore derives one semantic relation, DEPENDS_ON, directed from *dependent* to *dependency* ("if the target fails, the source is impacted"), by the six rules of Table 3. This derivation is what makes a component's dependents countable, and counting them on the derived graph is the strongest ranker in this study (Section 7.1). Its weight $w \in (0, 1]$ expresses how likely a disruption of the dependency is to reach the dependent.

Rules 1 and 2 combine the topics T joining a pair by probabilistic union rather than maximum [80, 81, 82], so parallel failure paths always increase coupling. Rule 5 uses the harmonic mean $H(x, y) = 2xy/(x + y)$ [83], and Rules 3 and 4 lift dependencies to hosts by maximum.

Sequential cascades and simultaneous blasts. Rule 1 captures sequential cascades, in which a failed publisher starves subscribers through queues and topic buffers. Rule 5 captures simultaneous blasts, in

Table 3: The six **DEPENDS_ON** logical dependency projection rules.

Rule	Dependency Category	Structural Pattern (Dependent $\rightarrow$ Dependency)	Derived Weight (w)
1	app_to_app	Subscriber $\rightarrow$ Publisher (via shared Topic, incl. transitive USES)	$1 - \prod_{t \in T} (1 - w(t))$
2	app_to_broker	Publisher/Subscriber $\rightarrow$ Broker routing its topics	$1 - \prod_{t \in T} (1 - w(t))$
3	host_to_host	Host $\rightarrow$ Host (lifted from inter-host app dependencies)	$\max_{u \in \text{hosted}(h_1), v \in \text{hosted}(h_2)} w_{\text{DEPENDS_ON}}(u \rightarrow v)$
4	host_to_broker	Host $\rightarrow$ Broker (lifted from hosted app dependencies)	$\max_{u \in \text{hosted}(h)} w_{\text{DEPENDS_ON}}(u \rightarrow b)$
5	app_to_lib	Application $\rightarrow$ Shared Library it USES	$H(w_V(\text{app}), w_V(\text{lib}))$
6	broker_to_broker	Broker $\leftrightarrow$ Broker (shared physical fault-domain colocation, symmetric)	$w_V(\text{host})$

which a crashed library or host takes down every consumer at once. Untyped graphs collapse the two into indistinguishable edges. Rule 6, the only symmetric rule, joins brokers colocated on a host, which share its failure domain. Figure 2 shows both mechanisms on a seven-entity example.

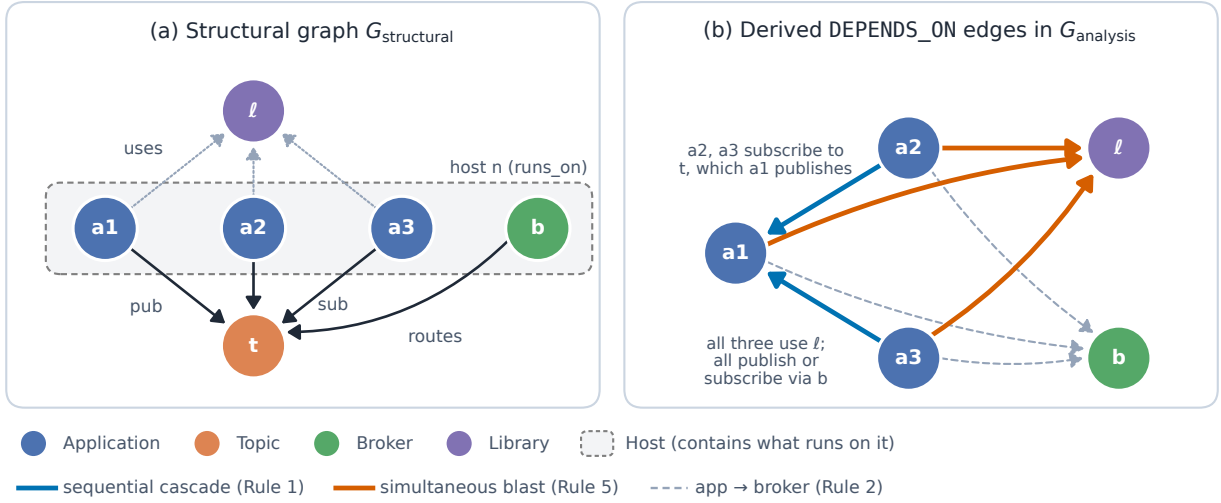


Figure 2: Running example. (a) Three applications share topic t (routed by broker b) and library l , and all run on host n . No structural edge joins two applications. (b) The derived **DEPENDS_ON** edges make the hidden dependencies explicit: the subscribers a_2, a_3 depend on the publisher a_1 (Rule 1, a sequential cascade through the topic), every application depends on l (Rule 5, a simultaneous blast if l fails), and each application depends on the broker routing its topic (Rule 2). Simulation oracles run on view (a) only; predictors read view (b).

3.3. Dual Graph Views

The **structural graph** $G_{\text{structural}}$ is the raw deployment topology. The **analysis graph** G_{analysis} adds the derived, QoS-weighted **DEPENDS_ON** edges and the code metrics (Figure 2). All predictor features are computed on G_{analysis} , while simulation oracles run only on $G_{\text{structural}}$ (Section 4.4); Rule 6 therefore cannot influence any label.

3.4. Typed Node Feature Encoding

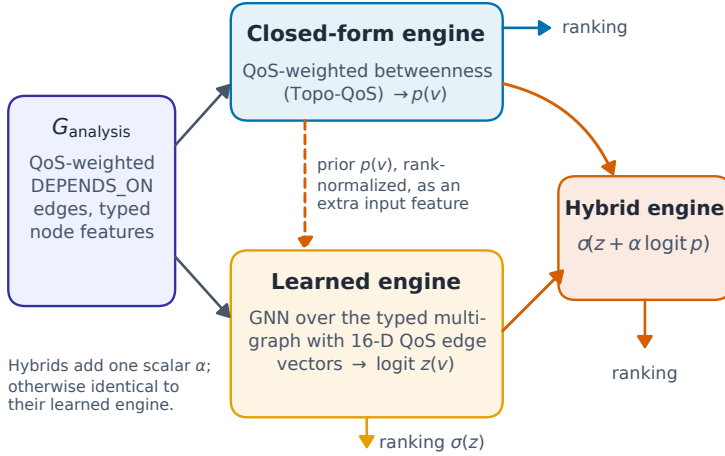
Both the predictive pathway (Section 4) and the explanation layer (Section 5) read the same typed node properties from G_{analysis} : the predictor projects them per entity type before message passing, the explanation layer aggregates them into a quality profile. All five entity types share a deterministic 18-dimensional base block of topological metrics, each normalized to $[0, 1]$ within its graph so that raw graph size does not drive cross-scenario transfer. The block comprises PageRank and Reverse PageRank; betweenness, closeness and eigenvector centrality; in- and out-degree; clustering; articulation and bridge scores; the node QoS weight and QoS-weighted in- and out-degree; multi-path coupling; path complexity; fan-out criticality; and the Connectivity Degradation Index (CDI), which removes each node and measures the resulting connectivity loss (full schema: Supplementary Section S11). Type-specific blocks extend the vector to 19–25 dimensions: code metrics and CQP for Applications, reverse-USES blast radius for Libraries, queue capacity for Brokers, publisher/subscriber counts and QoS criticality for Topics, and CPU and memory for Hosts. PageRank,

Reverse PageRank, betweenness and eigenvector centrality are computed on the QoS-weighted projection. They therefore carry QoS information into every predictor, including the “QoS-off” arms, which lack only the explicit QoS edge channel and QoS node columns. Because these topological summaries are available to any scorer, closed-form engines are genuine competitors rather than strawmen. CDI, at $O(|V|^2 + |V||E|)$, dominates analysis cost (Section 7.4).

4. Ranking Engines and Ground Truth

SaG ranks components with three kinds of engine. The **closed-form engine** Topo-QoS is QoS-weighted betweenness on the dependency projection (Section 6.2). The **learned engines** are graph neural networks over the typed multigraph (this section). The **hybrid engines** are learned engines that correct the closed-form score (Section 7.1.1). All are trained or scored against simulation oracles that run on a separate graph view (Sections 4.3–4.4). Figure 3 shows how the three engines relate and how they are evaluated. Full hyperparameters and training commands are on the experiment pages of the replication repository (Section 6.1).

(a) Three ranking engines, one analysis graph



(b) Ground truth and evaluation

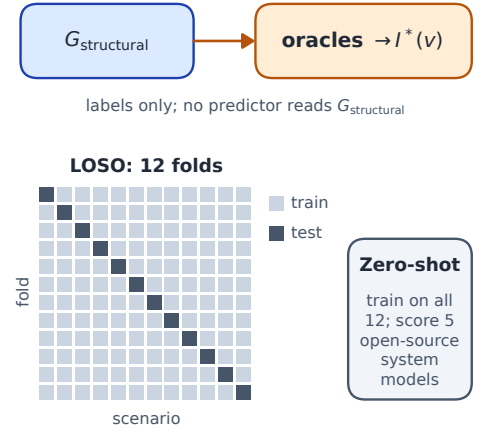


Figure 3: (a) SaG’s three ranking engines read the same analysis graph. The closed-form engine scores QoS-weighted betweenness $p(v)$; the learned engine outputs a logit $z(v)$. A hybrid engine gives the learned engine $p(v)$ as an extra input feature and adds a learned correction to it on the logit scale, $\sigma(z + \alpha \logit p)$, with one learnable scalar α . (b) Ground truth comes from simulation oracles on the structural graph, which no predictor reads. Engines are evaluated by leave-one-scenario-out cross-validation over twelve synthetic architectures (each row trains on eleven and tests on the held-out one) and zero-shot on five open-source system models.

4.1. Heterogeneous Graph Transformer

The learned engine HGT-QoS is a three-layer Heterogeneous Graph Transformer (HGT) [75] in PyTorch Geometric [84], with hidden dimension $D = 64$ and $H = 4$ heads. Entity-specific projections map raw features $x_v \in \mathbb{R}^{19-25}$ (Section 3.4) into the hidden space, $h_v^{(0)} = \text{LayerNorm}(\text{GELU}(W_{\tau(v)}x_v))$. For each meta-relation $\langle \tau(u), \phi(e), \tau(v) \rangle$, attention uses type-parameterized keys $K(u) = h_u W_K^{\tau(u)}$, queries $Q(v) = \tilde{h}_v W_Q^{\tau(v)}$ and values $V(u) = h_u W_V^{\tau(u)}$, scaled by a learned per-meta-relation prior $\mu_{\langle \tau(u), \phi(e), \tau(v) \rangle}$. That prior is the relation-typing mechanism that RQ2 tests. Message passing runs over both G_{analysis} and its transpose, to capture downstream starvation and upstream backpressure, with residual connections, dropout 0.10 and layer normalization.

4.1.1. QoS Edge Encoding (16-D)

Each directed edge carries a vector $e_{uv} \in \mathbb{R}^{16}$. Index 0 is the coupling weight $w_E(e)$ (Section 3.2), index 1 the normalized count of simple paths through e , indices 2–8 a one-hot of the seven relation types, and indices 9–15 the middleware QoS parameters on PUBLISHES_TO/SUBSCRIBES_TO edges, zero elsewhere. Six QoS dimensions are active in our corpus: reliability, durability, priority, a flag for edges whose QoS departs from the scenario’s modal profile, and a deadline pair (active flag and log-deadline, populated on 463 of 615 topics). A seventh, max-blocking time, is reserved for hard real-time profiles and is zero throughout. The encoding is projected and added to the target representation before attention, $\tilde{h}_v = h_v + W_{\text{edge}}e_{uv}$.

4.2. Prediction Head and Training Objective

A composite head predicts cascade impact, $\hat{I}^*(v) = \sigma(\text{MLP}_C(h_v \parallel \hat{a}_1(v) \parallel \hat{a}_2(v)))$. The two auxiliary heads $\hat{a}_1, \hat{a}_2$ act only as learned feature enrichment: $\hat{a}_1$ is supervised on I^* and $\hat{a}_2$ is unsupervised. The optimized objective combines regression with listwise and pairwise ranking:

$$\mathcal{L} = \text{MSE}(\hat{I}^*, I^*) + 0.5 \cdot \text{MSE}(\hat{a}_1, I^*) + 0.3 \cdot \mathcal{L}_{\text{rank}} + 0.1 \cdot \mathcal{L}_{\text{pairwise}}, \quad (5)$$

where $\mathcal{L}_{\text{rank}}$ is ListMLE [85] over the ground-truth permutation π ,

$$\mathcal{L}_{\text{rank}} = -\frac{1}{N} \sum_{i=1}^N \left(\hat{s}_{\pi_i} - \log \sum_{j=i}^N \exp(\hat{s}_{\pi_j}) \right), \quad (6)$$

and $\mathcal{L}_{\text{pairwise}}$ is a margin-ranking loss ($\gamma = 0.05$) over pairs whose true impacts differ by more than γ . A general form with a maintainability term and a consistency term tying the heads to the explanation layer exists but is switched off throughout, so the learned engines and the explanation layer share no parameters (Supplementary Section S20).

Models are trained with AdamW ($\eta = 3 \times 10^{-4}$, weight decay 10^{-4}) under cosine warm restarts for up to 300 epochs, with early stopping at patience 30 on an inner validation split. Five seeds {42, 123, 456, 789, 2024} are used throughout. Architectural hyperparameters follow conventional HGT values, and the loss coefficients were set by judgment. Neither was tuned on any evaluation split, so every learned arm is compared untuned.

4.3. Ground-Truth Simulation Oracles

Ground truth comes from failure simulations over the raw structural multigraph $G_{\text{structural}}$. Two oracles are used: the cascade-reachability oracle $I^*(v)$ is the ranking target of every research question, and a behavioral queue-flow oracle $I_{\text{dyn}}(v)$ checks its convergent validity.

Primary target, $I^*(v)$. The oracle crashes component v , propagates the outage through dependent topics, brokers and links by breadth-first traversal, and returns the mean fractional feed loss over the intact graph’s subscriber population. A topic’s feed loss is the fraction of its publishers that failed. It is scaled by a declared QoS severity ladder ($\times 1.2$ RELIABLE, $\times 1.15$ high priority, $\times 1.05$ medium) and clamped to $[0, 1]$. Five seeds break ties in propagation order, and $I^*(v)$ is their mean. It is reproducible from a fixed seed set, as CI gating requires.

How much QoS is in this label. The ladder reads reliability and priority only, but that does not bound the label’s QoS content much: disabling QoS scaling entirely leaves the Application ordering nearly intact — mean Spearman $\rho = 0.965$ against the ladder across the twelve folds (range 0.891–0.999) — and substituting a durability-aware $w(t)$ scaling moves it less still ($\rho = 0.977$). The top- K set is the sensitive construct: ladder and topology-only labels agree at mean Jaccard 0.678, so QoS changes *which* components are named critical rather than their order. I^* is therefore a near-topological target, which bounds what any QoS-encoding result can be credited with (Section 7.2).

Convergent-validity probe, $I_{\text{dyn}}(v)$. A SimPy [86] message-flow simulation of emission rates, stochastic latencies and broker buffer saturation returns the drop in delivered message rate to surviving consumers. It agrees with I^* at $\rho = 0.627$, which is substantial but below I^* ’s own seed-to-seed test–retest of 0.811–1.000, so the two measure related but distinct constructs (Supplementary Section S9). Topic criticality is a predictor input, so it is masked out of both oracles’ severity terms. Results established against one oracle are never transferred to the other.

4.4. Input-Label Independence Guarantee

Features are built only from G_{analysis} : static topology, code metrics and declared QoS. Labels are computed only on $G_{\text{structural}}$ by the simulation oracles. No simulation output or runtime telemetry is exposed as a predictor input, and a CI test enforces the separation (`tests/test_independence_guarantee.py`). This rules out circular feature construction. It does not make features and labels independent of the topology they both derive from: $I^*(v)$ is a reachability functional of the same architecture. The learning task is therefore to combine pre-computed structural cues into a ranking that matches the simulator, which is why the closed-form engine is a strong reference.

5. The Explanation Layer: Standards-Grounded Criticality Attribution

A ranking says where risk is highest, not how to reduce it. A component may be critical because it is an unreplicated single point of failure, an error-propagating cascade hub, or a high-coupling maintainability bottleneck. Each of these calls for a different intervention: replication, circuit breakers, or decoupling. The explanation layer attributes these causes after ranking. It reads the same node properties (Section 3.4), shares no parameters with the engines, and is not used as a ranker; its ranking correlation is reported for reference in Supplementary Sections S30 and S7.

5.1. Grounding in ISO/IEC Standards

Following ISO/IEC 25010:2023 [60] and ISO/IEC 25019:2023 [61], criticality is profiled along **Reliability** (R), split into **Fault Tolerance** (FT) and **Availability** (A), and **Maintainability** (M). FT captures error-cascade potential and informs circuit breakers and redundancy. A captures structural single points of failure and informs replication. M captures coupling and code-level complexity and informs decoupling and refactoring. Safety and security, which need hazard logs, are out of scope.

5.2. Composite Quality Score

Figure 4 summarizes the layer. All metrics are rank-normalized to $[0, 1]$ within the graph and combined with AHP-derived weights [64]:

- $FT(v) = 0.45 \cdot \text{RPR}(v) + 0.30 \cdot \text{Deg}_{\text{in}}(v) + 0.25 \cdot \text{CDPot}_{\text{enh}}(v)$, over Reverse PageRank, normalized in-degree and normalized cascade depth on G_{analysis}^T ;
- $A(v) = 0.25 \cdot \text{AP}_c^{\text{dir}}(v) + 0.20 \cdot \text{QSPOF}(v) + 0.20 \cdot \text{BR}(v) + 0.25 \cdot \text{CDI}(v) + 0.10 \cdot w(v)$, over directed articulation severity, QoS-weighted SPOF severity, bridge ratio, the Connectivity Degradation Index and the node QoS weight;
- $R(v) = r_{\text{FT}} \cdot FT(v) + (1 - r_{\text{FT}}) \cdot A(v)$ with $r_{\text{FT}} = 0.36$;
- $M(v) = 0.35 \cdot \text{BT}(v) + 0.30 \cdot w_{\text{out}}(v) + 0.15 \cdot \text{CQP}(v) + 0.12 \cdot \text{CouplingRisk}_{\text{enh}}(v) + 0.08 \cdot (1 - \text{CC}(v))$, over betweenness, QoS-weighted efferent coupling, code-quality penalty, coupling risk and clustering.

The composite is $Q(v) = 0.80 \cdot R(v) + 0.20 \cdot M(v)$, and an ISO/IEC 25019 context-of-use vector can reweight R and M . Intra-dimension weights are shrunk towards a uniform prior ($\lambda = 0.70$). If $Q(v)$ is used to rank, a fully uniform prior is better (0.319 vs. 0.200; Supplementary Section S1). The AHP matrices and their consistency diagnostics are in Supplementary Section S4. Components above the Tukey upper fence of Q are flagged CRITICAL (mean 4.2% of components). High A with low FT indicates a single point of failure that needs replication, while high FT indicates a cascade hub that needs circuit breakers (example card: Supplementary Section S19). The layer names the remediation class; generating candidate repairs and verifying them counterfactually is the subject of companion work on prescriptive remediation and is not evaluated here.

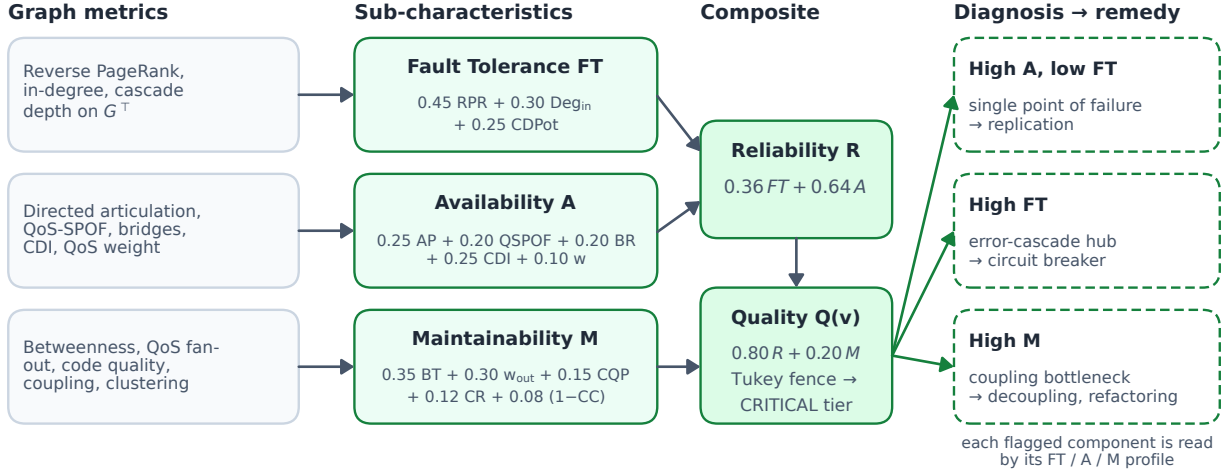


Figure 4: The explanation layer. Rank-normalized graph metrics feed the ISO/IEC 25010 sub-characteristics Fault Tolerance, Availability and Maintainability (CR: coupling risk; CC: clustering coefficient), which combine into Reliability and the composite $Q(v)$. A component above the Tukey fence of Q is flagged, and its $FT/A/M$ profile names the remediation class.

6. Experimental Setup

6.1. Corpus and Replication Package

The corpus comprises 2,812 components across seventeen architectures (Table 4). Twelve synthetic topologies form the LOSO folds. They span autonomous vehicles, financial trading, healthcare, industrial SCADA, smart-city IoT, telecom RAN, logistics, gaming, microservices, enterprise integration and air-traffic management. All twelve come from one generator family, as do their code metrics, so LOSO measures transfer across configurations of that generator. Each regenerates byte-identically from a committed configuration, and CI verifies this against a SHA-256 manifest.

Table 4: Evaluation corpus. The twelve synthetic topologies are the LOSO folds; the five open-source system models are excluded from all training and used only for zero-shot transfer (Section 7.3). Counts are read from the committed topology files and verified in CI; per-scenario composition: Supplementary Section S13.

Dataset	$ V $	$ V_{app} $	Topics	Brokers	Hosts	Libs	$ E $
Synthetic evaluation scenarios (11)	2,387	1,295	588	60	194	250	10,657
Synthetic case study — ATM (1)	74	26	27	5	8	8	261
Synthetic subtotal (12 LOSO folds)	2,461	1,321	615	65	202	258	10,918
Open-source system models (5)	351	141	120	16	32	42	700
Total	2,812	1,462	735	81	234	300	11,618

The five open-source systems are hand-authored models. Autoware.universe (ROS 2), EdgeX Foundry, Home Assistant, and two meshes modelled after Online Boutique and Train-Ticket were each written by one author as typed multigraphs from public documentation (`saag/adapters/realworld_adapter.py`). They are not mechanical extractions. Brokers, QoS profiles, code metrics and host specifications are partly assumed. Two models depart materially from their originals: the Online Boutique model is a 22-application pub-sub mesh with four brokers, whereas the original is about eleven gRPC services with no broker, and the Train-Ticket model represents its service-discovery server as a broker. Neither contains a synchronous call edge. RQ3 therefore tests transfer to independently authored architecture models, not to deployed systems.

Replication package. Datasets, harnesses, checkpoints and result artifacts are archived on Zenodo (see Data Availability). The public repository documents each experiment: its protocol, hyperparameters, `make` target, artifacts and the supplementary section holding its extended results (<https://github.com/onuralpyigit/software-as-a-graph/tree/jss-submission-v4/docs/research/jss/experiments>).

6.2. Predictors

SaG’s training-free engines on the dependency graph (**Topo-QoS**, **InDeg**, **Reach**), learned engines (**HGT-QoS**, **GAT-QoS**), hybrid engines and the learners on the dependency graph are compared against unweighted centrality (**Topo**) and untyped GNNs (Table 5). On a GNN, **-QoS** means the 16-D QoS edge vector (Section 4.1.1) together with three QoS node columns (w , w_{in} , w_{out}), and **Hybrid-X** is engine X corrected by the closed-form prior. **GAT** and **GAT-QoS** are untyped GATs matched to HGT in parameter budget, and **GAT-QoS** reads the same 16-D edge vector as **HGT-QoS**, so the four learned models form a 2×2 over typing (GAT vs. HGT) and the QoS channel. Smaller and projection-based variants that the study also ran are listed in Supplementary Section S29. The learned and hybrid engines ingest the native typed multigraph under **LOSO**, and the **-P** learners the Application–Library **DEPENDS_ON** graph, with bit-identical node features and labels. In QoS-off arms every edge weight is 1 and the QoS node columns are zeroed, though four centralities still carry QoS (Section 3.4).

Table 5: Predictors reported in this paper. **-QoS**: QoS-weighted distances (**Topo**) or the 16-D QoS edge vector (GNNs); **-P**: trained on the dependency graph; **Hybrid-X**: engine X corrected by the **Topo-QoS** prior (**Hybrid-GAT-P**: by the **InDeg** prior). Each learner on the dependency graph keeps its raw-multigraph counterpart’s architecture, edge features and parameter budget (**HGT-P-QoS** at width 100: 430,680 parameters). Every predictor run in the study, with the earlier names used by the registered plan and the artifacts: Supplementary Section S29.

Predictor	Evaluation Substrate	Typing	Edge Features	Parameters	Trained?	Empirical Role
<i>Training-free</i>						
Topo	<i>G_{analysis}</i> (Application layer)	No	None	0	No	Standard centrality baseline
Topo-QoS	<i>G_{analysis}</i> (App-Lib DEPENDS_ON)	No	Scalar $w(e)$	0	No	SaG closed-form engine
Reach	<i>G_{analysis}</i> (App-Lib DEPENDS_ON)	No	None	0	No	Transitive dependents
InDeg	<i>G_{analysis}</i> (App-Lib DEPENDS_ON)	No	None	0	No	Direct dependents
<i>Learned: typing $\times$ QoS channel at matched parameter budget</i>						
GAT	Native Multigraph	Homogeneous	None	437,496	Yes	Untyped, no QoS channel
GAT-QoS	Native Multigraph	Homogeneous	16-D QoS Vector	429,992	Yes	Untyped SaG learned engine
HGT	Native Multigraph	Heterogeneous	Relation 1-hot; $w(e)=1$	434,620	Yes	Typed, no QoS channel
HGT-QoS	Native Multigraph	Heterogeneous	16-D QoS Vector	434,620	Yes	Typed SaG learned engine
<i>Hybrid engines: a learned engine corrected by the Topo-QoS prior</i>						
Hybrid-HGT	Native Multigraph	Heterogeneous	16-D QoS Vector	434,941	Yes	HGT-QoS + prior
Hybrid-GAT	Native Multigraph	Homogeneous	16-D QoS Vector	431,433	Yes	GAT-QoS + prior
<i>Learned on the dependency graph (Amendment 9)</i>						
GAT-P , GAT-P-QoS , HGT-P-QoS , Hybrid-GAT-P	App-Lib DEPENDS_ON	As counterpart	As counterpart	As counterpart	Yes	GAT , GAT-QoS , HGT-QoS , Hybrid-GAT on the dependency graph

Closed-form scores. **Topo-QoS** is computed on the Application–Library **DEPENDS_ON** graph (Rules 1 and 5), because on the raw multigraph messages route through topics and brokers and Application betweenness vanishes; the registered comparator **Topo** reads betweenness from the analysis stage’s application-layer graph:

$$\text{Topo}(v) = 0.6 \cdot \text{BT}(v) + 0.4 \cdot \text{AP}(v), \quad \text{Topo-QoS}(v) = 0.6 \cdot \text{BT}_w(v) + 0.4 \cdot \text{AP}(v), \quad (7)$$

where **BT** is normalized betweenness, BT_w is betweenness over edge distances $d(e) = 1/(w(e) + 10^{-6})$, so strongly coupled edges attract shortest paths, and **AP** flags articulation points. In the evaluated implementation the articulation term reads zero for every node, so the reported scores rank exactly as betweenness and QoS-weighted betweenness. Restoring it lowers both (Supplementary Section S22), so the evaluated form is the stronger reference and is kept as registered. The two therefore differ in substrate as well as in weighting, and the registered controls of Section 7.1 separate the two.

Dependency counts. $\text{InDeg}(v)$ is the number of direct dependents of v on the Application–Library **DEPENDS_ON** graph, and $\text{Reach}(v)$ the number of its transitive dependents, normalized by $|V| - 1$. Both were registered in Amendment 7 before their results existed. No predictor reads $G_{\text{structural}}$ (Section 4.4).

6.3. Metrics, Protocols and Statistics

Population. Every predictor in a table is scored on the same node population, the Application set V_{app} . Pooling entity types conflates distinct base rates (Supplementary Section S6).

Metrics. Ranking is measured by Spearman ρ against $I^*(v)$. Critical-set identification is measured by $\text{Overlap}@K$, the fraction of the true top- K recovered by the predicted top- K with $K = \text{round}(0.20|V_{\text{app}}|)$, at which top- K precision, recall and F_1 coincide. PR-AUC, $F_1@7$ and nDCG@10 are reported in Supplementary Section S15.

Protocols. Under **LOSO**, models train on eleven scenarios and are tested zero-shot on the twelfth, over all 12 folds and five seeds, with equal 3-layer depth and inner-split early stopping. Under *zero-shot transfer*, models trained on all twelve scenarios are evaluated without fine-tuning on the five system models. In-distribution node-split results are in Supplementary Section S17.

Statistics. We use paired Wilcoxon signed-rank tests [87] and bootstrap 95% CIs ($B = 2,000$) over folds [88, 89]; with 12 folds, the smallest attainable two-sided p is 0.00049. Folds share ten of eleven training scenarios, so the tests are anti-conservative [90], and we read all p -values as nominal.

Registered analysis plan. Before the twelve-fold harness produced any result, we registered the primary contrast, HGT-QoS vs. Topo-QoS, together with HGT vs. Topo-QoS and Holm correction across the two. We call it *registered* rather than pre-registered, because the plan is a file in our repository with no third-party timestamp. Three amendments registered further confirmatory contrasts, each before its run and each Holm-corrected within its own family: the matched 2×2 with its capacity and directionality controls (Amendment 2), Hybrid-HGT (Amendment 5) and Hybrid-GAT (Amendment 6). Three more registered exploratory families before their runs, each with its own Holm correction and decision rules: the dependency counts and QoS controls (Amendment 7), the learners on the dependency graph (Amendment 9) and the value of the derivation (Amendment 10). Every other contrast is exploratory. Because the sequence was adaptive (Amendment 6 followed Amendment 2’s result), we also pool all thirteen registered contrasts under one Holm correction (`reproduce/omnibus_holm.py`). Both hybrid primaries remain significant (Hybrid-GAT $p_{\text{omni}} = 0.019$, Hybrid-HGT $p_{\text{omni}} = 0.041$), and no other registered contrast reaches $\alpha = 0.05$ ($p_{\text{omni}} \geq 0.46$). Supplementary Section S24 lists all ten amendments with dates and outcomes.

7. Results

All results are reported on the Application population (V_{app}) against the primary oracle $I^*(v)$, under the input-label independence guarantee (Section 4.4). Per-fold results, secondary strata and extended protocol notes are in the Supplementary Material and the experiment pages of the replication repository (Section 6.1). Figure 5 summarizes the main findings.

7.1. RQ1: SaG’s Engines Against Structural Baselines

Summary. On SaG’s derived dependency graph, counting a component’s dependents ranks its cascade impact as well as any engine in the study: **InDeg** reaches $\rho = 0.764$ and beats closed-form centrality (**Topo-QoS**, 0.553) on all twelve held-out architectures (+0.211, Holm $p = 0.002$). Deriving the dependencies is what makes the count work: counting raw connections reaches 0.199 (+0.565 for **InDeg**, 12/12). A graph neural network that reads the dependency graph reaches the same level (**GAT-P-QoS**, 0.748). The hybrid engines significantly outperform closed-form centrality under their registered rules (+0.103 and +0.130, each on 11/12 folds, Holm $p \leq 0.0068$).

Each of the twelve folds holds out one scenario and trains on the remaining eleven, and every predictor is scored on the same Application node set (26 to 300 nodes, K between 5 and 60). Table 6 reports the training-free engines on the dependency graph, SaG’s learned and hybrid engines on the raw multigraph, and the same learners on the dependency graph.

Counting dependents on the derived graph ranks impact well. **InDeg**, the number of components that depend on v in the `DEPENDS_ON` graph, reaches $\rho = 0.764$ and beats **Topo-QoS** on every fold; **Reach**, the number of transitive dependents, reaches 0.732 (11/12). Both were registered in Amendment 7 before their results existed. For an Application, **InDeg** is the number of distinct subscribers to the topics it publishes: publish-subscribe afferent coupling [52, 53, 54], which the derivation makes computable from an architecture model. Amendment 10 shows that the derivation is what makes the count predictive. Counting a component’s raw connections in the multigraph scores 0.199, and counting the topics it publishes 0.431; **InDeg** beats both on all twelve folds (+0.565 and +0.334, Holm $p = 0.0015$). Transitive reach also profits from the derived library rule: without Rule 5, **Reach** falls by 0.058 (9/12, Holm $p = 0.0068$).

Learners that read the dependency graph match the count. The same graph neural networks, trained on the dependency graph instead of the raw multigraph, reach $\rho = 0.748$ (**GAT-P-QoS**), statistically level with **InDeg** (-0.017 , Holm $p = 1.000$) but not above it. Anchoring the same learner on the **InDeg** score adds nothing further (Hybrid-GAT-P, -0.006 against **InDeg**; Table 7). Section 7.2 shows why reading the dependency graph matters.

Both registered hybrids significantly outperform closed-form centrality. Hybrid-HGT reaches $\rho = 0.657$ (+0.103, Holm $p = 0.0068$) and Hybrid-GAT reaches $\rho = 0.683$ (+0.130, CI [+0.075, +0.190],

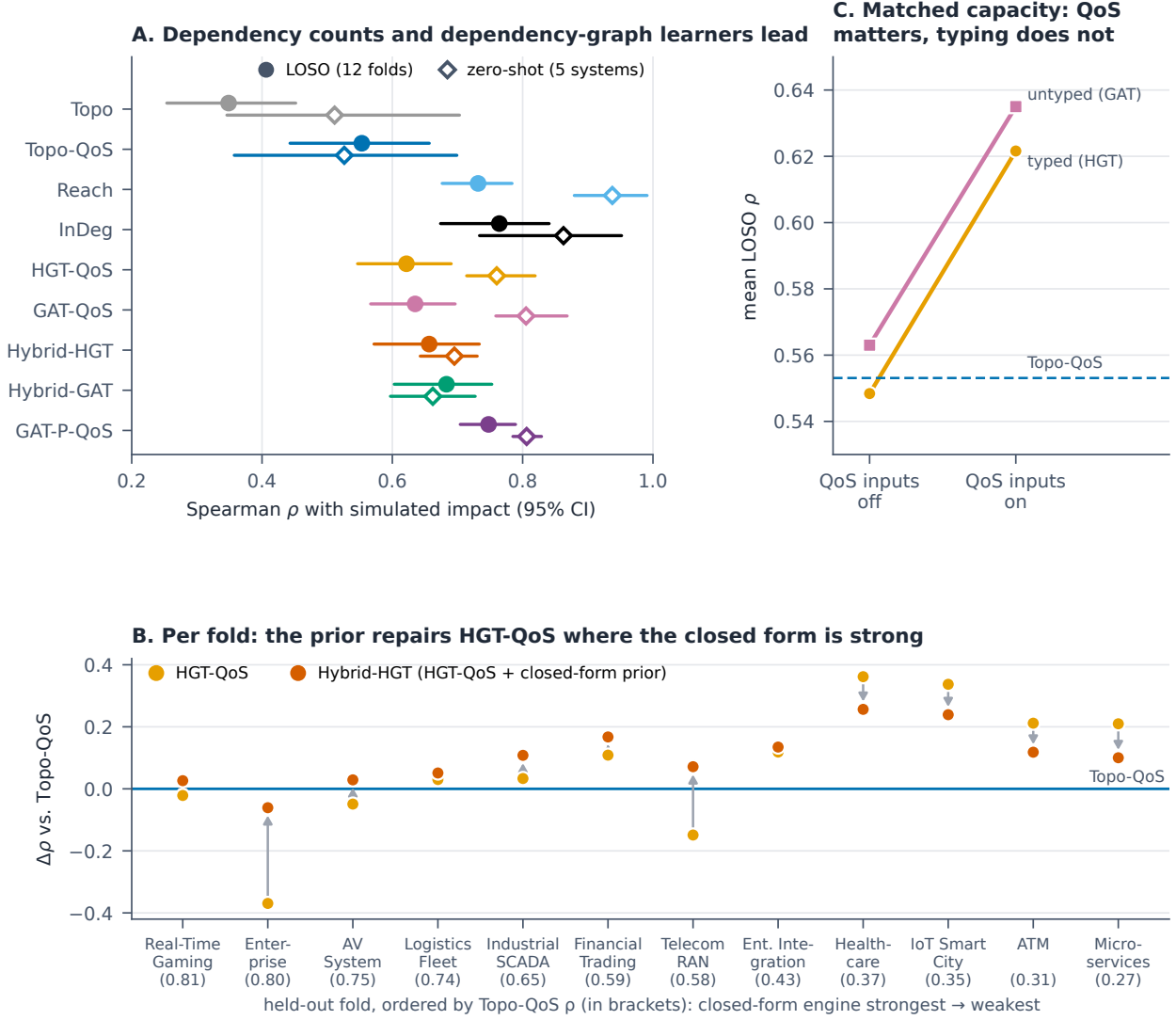


Figure 5: Main results at a glance, Application population. (A) Mean Spearman ρ with 95% bootstrap intervals under LOSO (filled circles; Table 6) and zero-shot on the five system models (open diamonds). Counting dependents on the derived dependency graph (**InDeg**, **Reach**) gives the highest means both on unseen synthetic architectures and on the system models, and the learner that reads the dependency graph (bottom row) reaches the same level on unseen synthetic architectures. (B) Per held-out fold, the gain of **HGT-QoS** and of **Hybrid-HGT** over **Topo-QoS**; the arrow shows what the closed-form prior changes. It removes the learned engine’s losses where the closed-form engine is strongest (Enterprise, Telecom RAN) and trims its largest gains where it is weakest. (C) Cell means of the capacity- and channel-matched 2×2 (Table 8): the QoS inputs raise both models by about 0.07, while the typed and untyped lines stay together.

Table 6: Main results under LOSO (twelve synthetic architectures, Application population; learned engines: five seeds, CPU sweeps) and zero-shot on the five hand-authored system models (protocol of Table 9). $\Delta\rho$ is paired by fold against the registered closed-form engine **Topo-QoS**, with a bootstrap 95% CI ($B = 2,000$) and a two-sided Wilcoxon test. p_{Holm} is given within each registered family: the hybrids’ (Amendments 5 and 6) and the dependency counts’ (Amendment 7). [‡]Descriptive: Amendment 9 registers the dependency-graph learners’ contrasts against **InDeg**, **Reach** and their raw-multigraph counterparts (Table 7). [†]Scored by the Amendment 7 harness on the same models and oracle settings; in that harness **Topo-QoS** scores 0.582 rather than 0.526 (Supplementary Table S36). Per-fold values: Supplementary Sections S23, S32 and S33; the registered GPU sweep of **HGT-QoS**: Section S30.

Predictor	LOSO, twelve synthetic architectures					Five system models	
	Mean ρ [95% CI]	$\Delta\rho$ vs Topo-QoS [95% CI]	Won	p (p_{Holm})	Overlap@K	ρ [95% CI]	PR-AUC
<i>Training-free, on the dependency graph</i>							
Topo	0.349 [0.254, 0.452]	−0.204 [−0.286, −0.122]	0/12	0.0005	0.366	0.511 [0.346, 0.703]	0.474
Topo-QoS	0.553 [0.443, 0.657]	—	—	—	0.388	0.526 [0.357, 0.699]	0.474
Reach	0.732 [0.674, 0.782]	+0.178 [+0.088, +0.268]	11/12	0.0034 (0.0068)	0.341	0.938 [†] [0.879, 0.991]	0.933 [†]
InDeg	0.764 [0.674, 0.840]	+0.211 [+0.132, +0.299]	12/12	0.0005 (0.0020)	0.506	0.863 [†] [0.734, 0.952]	0.752 [†]
<i>Learned and hybrid, on the raw multigraph</i>							
HGT-QoS	0.622 [0.547, 0.690]	+0.069 [−0.046, +0.174]	8/12	0.266	0.426	0.760 [0.714, 0.819]	0.713
GAT-QoS	0.635 [0.567, 0.696]	+0.082 [−0.046, +0.201]	7/12	0.233	0.438	0.805 [0.759, 0.868]	0.790
Hybrid-HGT	0.657 [0.572, 0.733]	+0.103 [+0.055, +0.152]	11/12	0.0034 (0.0068)	0.435	0.695 [0.643, 0.730]	0.602
Hybrid-GAT	0.683 [0.603, 0.753]	+0.130 [+0.075, +0.190]	11/12	0.0015 (0.0029)	0.450	0.662 [0.597, 0.727]	0.600
<i>Learned, on the dependency graph (Amendment 9)</i>							
GAT-P-QoS	0.748 [0.704, 0.789]	+0.195 [+0.100, +0.294]	10/12	0.0068 [‡]	0.454	0.806 [0.785, 0.829]	0.838

Holm $p = 0.0029$), each on 11 of 12 folds. Each meets the decision rule registered before its run, and both remain significant under one Holm correction pooled over all thirteen confirmatory contrasts of the study ($p_{\text{omni}} = 0.041$ and 0.019 ; Section 6.3). On their own, the raw-multigraph learned engines are on par with **Topo-QoS** (**HGT-QoS** $+0.069$, $p = 0.266$; **GAT-QoS** $+0.082$, $p = 0.233$; registered GPU sweep $+0.085$, Holm 0.303 ; Supplementary Section S30).

Why the hybrids work: the engines are complementary. **HGT-QoS** loses to **Topo-QoS** on four folds, all where the closed-form engine is strongest: Real-Time Gaming, Enterprise (0.426 vs. 0.795), AV System and Telecom RAN (0.427 vs. 0.576). Its largest gains come where the closed-form engine is weakest: Healthcare, IoT Smart City, ATM and Microservices, $+0.210$ to $+0.362$ (Figure 5B). The prior removes the failure mode: on Enterprise, Hybrid-HGT and Hybrid-GAT reach 0.735 and 0.768, and Telecom RAN turns from a loss into a win for both. The cost is a smaller gain on the weakest folds.

Where the closed-form gain comes from. The registered comparator **Topo** reads betweenness from the analysis stage’s application-layer graph, whereas **Topo-QoS** computes QoS-weighted betweenness on the Application–Library dependency graph. Three controls registered in Amendment 7 separate substrate from weighting (Supplementary Table S37). Unweighted betweenness on the same graph scores 0.591; fixing every topic weight to a constant scores 0.595; permuting QoS profiles across topics scores 0.559 (-0.006 against the declared profiles, $p = 0.73$). By decision rule R2 of Amendment 7, the $+0.204$ is therefore attributed to QoS-weighted dependency multiplicity on the Application–Library graph, not to the content of the declared contracts, and a QoS-independent control corpus shows the generator’s QoS–topology coupling to be part of that mechanism (R2’). This matches the label analysis of Section 4.3, where the oracle’s ordering is 96.5% topological. Declared contracts pay off for learned engines instead (Section 7.2), and oracles that express deadline misses, durability replay or priority inversion are where their content can be expected to matter for closed-form ranking.

Anchoring trades transfer for in-distribution accuracy. On the five independently authored system models, both hybrids stay well above closed-form centrality (0.695 and 0.662 vs. 0.511–0.526) but below the pure learned engines (0.760 and 0.805; Section 7.3). By the rule registered in Amendment 6, Hybrid-GAT therefore does not replace Hybrid-HGT as the recommended hybrid ($0.662 < 0.695$).

Label noise and inert components. Re-running the oracle across five seeds gives a test–retest rank correlation of 0.811–1.000 (median 0.982), well above every engine’s mean ρ . Between 21% and 52% of each held-out population carries zero simulated impact. In the registered sweep, restricting evaluation to components with positive impact halves every predictor’s correlation, learned or not, and leaves the method ordering unchanged (Supplementary Section S25).

7.1.1. How the Hybrids Are Built

Each hybrid gives a learned engine the rank-normalized **Topo-QoS** score as one extra input per Application and Library, and adds a learned correction to that score on the logit scale, $\hat{I}^*(v) = \sigma(z(v) + \alpha \logit(p(v)))$, with one learnable α (Figure 3a). Everything else matches the underlying engine. Hybrid-HGT is built on **HGT-QoS** (Amendment 5; 321 extra parameters) and Hybrid-GAT on **GAT-QoS** (Amendment 6; 1,441 extra parameters). Each was registered with its contrasts and decision rule before any run, with no setting tuned, and evaluated in its own CPU sweep with its comparators re-run in the same invocation.

7.2. RQ2: What Learned Engines Need

Summary. Learned engines need two things. The first is the dependency graph: the same networks gain +0.075 to +0.113 when they read it instead of the raw multigraph (11–12 of 12 folds, Holm $p \leq 0.021$), because on it every component receives messages from its dependents. The second is the declared coupling inputs: with capacity and edge-channel width matched, QoS inputs raise learned ranking by +0.073 (10 of 12 folds), while relation-specific weights add nothing (typing main effect -0.014 , interaction $+0.001$).

Learning pays off when it reads the dependency graph. On the raw multigraph every relation points away from an Application, so the untyped networks score each Application from its own features, and HGT reaches it only through its reverse pass (Section 8.2). Amendment 9 moved the same learners, at the same parameter budgets and with bit-identical node features and labels, onto the **DEPENDS_ON** graph. There a 3-layer network’s receptive field at an Application is exactly the Application and its dependents within three hops, on all twelve architectures. Table 7 reports the registered contrasts. All three GAT arms beat their raw-multigraph counterparts, by +0.090 (**GAT-P**), +0.113 (**GAT-P-QoS**, 12/12) and +0.075 (Hybrid-GAT-P, 11/12), and the two QoS-reading arms reach the level of **InDeg** without exceeding it. Message passing now contributes beyond the node features: **GAT-P-QoS** beats the feature-only gradient-boosting control **GBM-Feat** by +0.106 (10/12, descriptive), where on the raw multigraph **GAT-QoS** was only level with it (-0.007). The typed HGT-P-QoS did not train stably on this graph (median within-fold seed spread 0.208, against 0.026 for **GAT-P-QoS**); its low mean reflects that failure rather than the graph, and it was reported as registered, untuned.

Table 7: Learners on the dependency graph (Amendment 9): LOSO mean ρ and the twelve registered contrasts, Holm-corrected within this exploratory family. Each arm keeps its raw-multigraph counterpart’s parameter budget (HGT-P-QoS at width 100: 430,680 parameters) and node features. Zero-shot on the five system models is descriptive. Per-fold values: Supplementary Section S33.

Learner on the dependency graph	LOSO ρ	vs InDeg	vs Reach	vs raw-multigraph counterpart	Zero-shot ρ
GAT-P	0.653	-0.111 (1/12, 0.017)	-0.079 (1/12, 0.009)	+0.090 vs GAT (11/12, 0.012)	0.830
GAT-P-QoS	0.748	-0.017 (4/12, 1.000)	$+0.016$ (6/12, 1.000)	+0.113 vs GAT-QoS (12/12, 0.006)	0.806
Hybrid-GAT-P	0.758	-0.006 (3/12, 0.881)	$+0.026$ (8/12, 1.000)	+0.075 vs Hybrid-GAT (11/12, 0.021)	0.792
HGT-P-QoS	0.514	-0.250 (0/12, 0.006)	-0.217 (0/12, 0.006)	-0.107 vs HGT-QoS (4/12, 0.881)	0.746

Relation typing at matched capacity. A first comparison against small untyped GATs (28,168 parameters against HGT’s 434,620, reading at most a scalar edge weight) credited typing with a large gain (+0.234 without QoS). That gain is an effect of capacity and channel width (Supplementary Sections S26 and S30), and the control registered in Amendment 2 removes both differences. **GAT** and **GAT-QoS** are untyped GATs at HGT’s parameter budget (437,496 and 429,992), and **GAT-QoS** reads the same 16-D edge vector as **HGT-QoS**, including the relation one-hot. All four matched arms ran in one CPU sweep, and the decision rule was fixed before any control result existed (Table 8).

QoS inputs are the working ingredient. At matched capacity, adding the QoS inputs raises ranking by +0.073 with or without typing, on 10 of 12 folds each time, and significantly for the untyped pair (+0.072, CI [+0.028, +0.109], $p = 0.016$). The QoS inputs also stabilize training: the median within-fold seed spread of the untyped pair falls from 0.083 to 0.010 (the small GATs and HGT show the same pattern; Supplementary Section S30). The Q factor switches two inputs together: the 16-D edge vector and three node columns holding each component’s declared coupling weights (w , w_{in} , w_{out}). An exploratory follow-up that separates them locates the gain in the node columns: removing them from **GAT-QoS** costs 0.095 (11 of 12 folds, Holm $p = 0.024$), whereas adding the edge channel alone to **GAT** gives -0.023 . The seed stabilization follows the same columns (median spread 0.010 with them, 0.136 without).

Table 8: The 2×2 with capacity and edge-channel width matched (Amendment 2): GAT ($\neg T \neg Q$, 437,496 parameters), HGT ($T \neg Q$, 434,620), GAT-QoS ($\neg T Q$, 429,992), HGT-QoS ($T Q$, 434,620). One CPU sweep, twelve LOSO folds, five seeds, Application population. Holm correction across the three orthogonal quantities; simple effects are descriptive. Cell means: 0.563, 0.548, 0.635, 0.622.

Quantity	Contrast	$\Delta\rho$	95% CI	Won	W	p	p_{Holm}
<i>Three orthogonal quantities, Holm-corrected across these three</i>							
Typing (main effect)	averaged over Q	-0.014	[-0.052, +0.023]	4/12	29.0	0.470	0.940
QoS channel (main effect)	averaged over T	+0.073	[+0.013, +0.120]	10/12	13.0	0.043	0.127
Typing $\times$ QoS interaction	difference of differences	+0.001	[-0.050, +0.042]	6/12	34.0	0.733	0.940
<i>Simple effects — descriptive, not separately corrected</i>							
Typing, QoS absent	HGT vs. GAT	-0.015	[-0.064, +0.033]	5/12	31.0	0.569	—
Typing, QoS present	HGT-QoS vs. GAT-QoS	-0.013	[-0.054, +0.026]	4/12	27.0	0.380	—
QoS channel, typing absent	GAT-QoS vs. GAT	+0.072	[+0.028, +0.109]	10/12	9.0	0.016	—
QoS channel, typing present	HGT-QoS vs. HGT	+0.073	[-0.002, +0.136]	10/12	19.0	0.129	—

Relation-specific weights add nothing once capacity is matched. HGT and HGT-QoS are within 0.015 of their untyped counterparts GAT and GAT-QoS and win only 4–5 of 12 folds against them. Because GAT-QoS receives each edge’s relation type as a feature, the precise finding is that relation-typed *parameters* add nothing beyond relation-typed *inputs*. The directionality control registered in Amendment 2 agrees: HGT-QoS-U, which removes HGT’s reverse pass and with it the only route by which messages reach an Application, reaches 0.632 against HGT-QoS’s 0.622 (registered contrast HGT-QoS vs. HGT-QoS-U -0.010, HGT-QoS ahead on 6/12 folds, $p = 0.91$; Section 8.2). The other registered control, a capacity-matched GAT that reads only the scalar weight $w(e)$, is level with GAT-QoS (Supplementary Section S24).

How much QoS the target can reward. $I^*(v)$ is a near-topological target: a topology-only relabeling recovers its ordering at mean $\rho = 0.965$, and QoS acts mainly at its top- K boundary (Section 4.3). On this target the QoS inputs therefore act largely as a coupling-strength signal. Oracles that express deadline misses, durability replay or priority inversion would let the encodings contribute contract semantics as well.

7.3. RQ3: Zero-Shot Transfer to Models of Open-Source Systems

Summary. The dependency structure SaG derives transfers without training: on independently authored models of five open-source systems, counting transitive dependents ranks impact at $\rho = 0.938$ [0.879, 0.991] and counting direct dependents at 0.863, and transitive reach keeps $\rho = 0.871$ on the components that actually propagate failures. Learned engines trained only on synthetic scenarios also transfer, far above closed-form centrality: HGT-QoS reaches 0.760 [0.714, 0.819] and GAT-QoS 0.805 [0.759, 0.868], against 0.511–0.526, and they nearly double its top- K critical-set overlap.

The five systems are hand-authored models of Autoware.universe (ROS 2), EdgeX Foundry and Home Assistant, plus meshes modelled after Online Boutique and Train-Ticket (Section 6.1). They were written independently of the scenario generator but are models rather than extractions, and they carry labels from the same oracles. The test is therefore transfer to independently authored topologies under simulated reachability. HGT-QoS and GAT-QoS were trained on all twelve synthetic scenarios and evaluated zero-shot at the same 3-layer, 300-epoch budget as every other learned result. No system model contributed gradients or checkpoint selection. Where a system declares no QoS manifest, standard middleware defaults (ROS 2 Best-Effort/Reliable, MQTT QoS 0/1) are applied uniformly across all predictors.

Dependency structure transfers without training. The dependency counts need no training data, so their transfer is immediate. Reach reaches $\rho = 0.938$ on the five system models (0.836–0.998 per system) and InDeg 0.863 (Table 6; Supplementary Table S36). These systems are small and have simple library structure: none of them has a Library that publishes or subscribes, so Rule-1-only reach equals Reach on every one of them (Amendment 10), and the counts separate inert from active components almost perfectly. They are therefore an easier ranking target than the synthetic folds, for every predictor.

Learned engines transfer far above closed-form centrality, independently of typing. Both learned engines lead closed-form centrality on 4 of 5 systems, with intervals that do not overlap its intervals (Table 6). The untyped GAT-QoS beats HGT-QoS on all five systems ($\rho = 0.805$ vs. 0.760). It also scores higher on Overlap@ K (0.519 vs. 0.470) and PR-AUC (0.790 vs. 0.713). Consistent with RQ2, what transfers is learning

Table 9: Zero-shot transfer to hand-authored models of five open-source systems: Spearman ρ against $I^*(v)$ on the Application population. Learned engines were trained on all twelve synthetic scenarios (3 layers, 300 epochs; five seeds, $\pm$ = spread over seeds). Training-free scores are deterministic and scored on identical labels and node sets. The last column restricts HGT-QoS to components with positive impact. All five models are encoded as publish-subscribe graphs; the grouping follows the paradigm of the *original* system. Identification metrics: Supplementary Table S14; bootstrap intervals and the active stratum for every predictor: Supplementary Section S27.

System model	$ V_{\text{app}} $	$n_{>0}$	Topo	Topo-QoS	HGT-QoS	GAT-QoS	HGT-QoS $\rho_{>0}$
<i>Originals are publish-subscribe systems</i>							
Autoware.universe (ROS 2)	32	19	0.307	0.378	0.716 ± 0.081	0.758 ± 0.019	+0.517
EdgeX Foundry (Industrial IoT)	22	10	0.534	0.534	0.793 ± 0.037	0.815 ± 0.055	+0.183
Home Assistant (Smart Home)	24	17	0.297	0.289	0.864 ± 0.063	0.925 ± 0.023	+0.702
<i>Originals are RPC systems (modelled as pub-sub)</i>							
Online Boutique (pub-sub model)	22	8	0.891	0.888	0.710 ± 0.070	0.750 ± 0.119	-0.031
Train-Ticket Booking Mesh	41	14	0.528	0.541	0.717 ± 0.096	0.777 ± 0.007	-0.192
Mean	—	—	0.511	0.526	0.760	0.805	+0.236

over SaG’s QoS-annotated graph, not relation-specific parameters. Identification separates the engines most clearly: top- K overlap averages 0.470 against 0.248 for the closed-form scores, and PR-AUC 0.713 against 0.474–0.521. On EdgeX, symmetric adapter-to-broker stars create betweenness ties that collapse closed-form triage entirely (Overlap@ K = 0.000).

Active components. Restricted to components that propagate failures, HGT-QoS keeps a positive mean correlation ($\rho_{>0}$ = +0.236) where closed-form centrality turns negative, and transitive reach keeps $\rho_{>0}$ = 0.871 (InDeg 0.321). Every learned interval spans zero at five systems, so the learned-versus-centrality comparison on this stratum is unresolved. Learners trained on the dependency graph (Amendment 9) transfer at 0.792–0.830, level with their raw-multigraph counterparts (Table 7). $\rho_{>0}$ is positive on the three models of publish-subscribe systems and non-positive on the two modelled after RPC systems. Because both RPC-derived models are encoded as publish-subscribe graphs, this split cannot be attributed to call-tree semantics (Section 8.4). An earlier, non-blind 2-layer configuration leaves every conclusion unchanged (Supplementary Section S29).

7.4. RQ4: Analysis Cost

Summary. Neural inference is effectively free (56 ms for a 2,000-node architecture, 0.02% of pipeline time). Cost is dominated by deterministic feature extraction, specifically the $O(|V|^2 + |V||E|)$ Connectivity Degradation Index, and it tracks the density of the derived dependency projection. Cold extraction takes 2–18 $\times$ (median 5.6 $\times$) as long as the in-process cascade simulation, so SaG’s advantage for the learned engines is avoiding staging infrastructure rather than saving CPU time. The strongest rankers are also the cheapest: deriving the dependency graph and counting dependents takes at most 0.12 s per scenario, including the 300-application Enterprise architecture, and needs none of the feature extraction.

Table 10: Per-stage latency of the inference pipeline across graph sizes (CPU, median of 3 runs; 5 for the forward pass). The analysis stage is stable across repeats (p10–p90 within 1% of the median), while the forward pass is dominated by interpreter and dispatch overhead; the 249-node row still carries first-call warm-up.

$ V $	$ E $	Analyze (s)	Graph $\rightarrow$ Tensor (s)	HGT Forward (ms)	Analyze : Forward	Forward p10–p90 (ms)
249	1,127	1.74	0.010	26.5	66 $\times$	13.0–34.4
499	2,402	8.32	0.022	16.4	509 $\times$	15.6–16.4
999	6,422	44.54	0.056	21.1	2,108 $\times$	19.1–36.5
1,998	19,301	239.34	0.157	56.2	4,259$\times$	43.8–57.8

At 2,000 components the HGT forward pass takes 56 ms against 239 s for structural analysis (Table 10). Because the analysis stage produces the node features the forward pass consumes, end-to-end evaluation of an unseen 2,000-component architecture takes about four minutes, of which the learned model is 0.02%. The 56 ms is the marginal cost of re-scoring an already-analysed graph. The dominant term is the Connectivity Degradation Index, computed for every node in the main connected component. That is a correctness

requirement: restricting CDI to articulation points leaves it identically zero in redundant multi-publisher topologies and makes $A(v)$ near-constant.

Across the corpus, the complete analysis gate (structural analysis plus 18 anti-pattern detectors) runs in 0.16–79.3 s per scenario. That is $2.0\text{--}17.7\times$ (median $5.6\times$) the five-seed cascade labeling sweep measured in the same session, and the gate is more expensive on all twelve scenarios. The premium tracks the size of the derived projection rather than component count; Enterprise, with 300 applications on 120 topics, is the maximum (Supplementary Section S28). On raw CPU time, direct simulation is therefore faster wherever its parameters are available. SaG additionally scores infrastructure components and dependency edges and avoids staging infrastructure. Training the four learned arms once took 7.7 CPU-hours, amortized over every later evaluation.

8. Discussion, Threats to Validity, and Limitations

8.1. Practical Consequences

The dependency graph carries the signal. The most robust result is that SaG’s derived dependency graph makes criticality legible to simple and learned analyzers alike. Counting a component’s dependents on it ranks cascade impact at $\rho = 0.764$, above closed-form centrality on every held-out architecture, and counting its transitive dependents ranks five independently authored system models at 0.938 without training. The derivation is what makes this work: the same components ranked by their raw connections score 0.199. Every other engine also improves when it reads the dependency graph: centrality on the Application–Library graph beats the application-layer baseline on all twelve folds (+0.204), and the same neural networks gain +0.075 to +0.113 over reading the raw multigraph. Practitioners therefore gain most from modeling their architecture with typed entities and deriving its dependencies, whichever engine they then run. Declared QoS contracts add value for learned engines (+0.073 at matched capacity), through declared-coupling node features rather than the edge channel.

Choosing an engine. Table 11 lists the recommended instruments. Dependency counting needs no training and runs in milliseconds, so it is the natural default for CI gates: **InDeg** for architectures like the synthetic corpus, **Reach** for systems unlike it. A learned engine should read the dependency graph (**GAT-P-QoS**), where it matches the counts; it is the component that can absorb richer inputs and oracles, such as the QoS semantics the current oracle does not express. **Topo-QoS**, the raw-multigraph learned engines and the hybrids are the evidence of Section 7, not recommended instruments: on this oracle the dependency counts match or exceed them both on held-out architectures and on the system models. Where a raw-multigraph engine is used nonetheless, the registered hybrids suit architectures resembling the training corpus, and Hybrid-HGT is the registered recommendation of the two because it transfers better. Because relation-specific weights add nothing at matched capacity, untyped networks are the simpler choice throughout; the untyped **GAT** without QoS inputs transfers at 0.831 zero-shot (Section 8.2). The explanation layer then names a remediation class for each flagged component.

Computational sustainability. Pre-deployment analysis avoids provisioning staging clusters for chaos sweeps [22, 24]; we state this as infrastructure avoidance, not a measured energy saving. At base SoC power (28 W), one pass of the analysis gate over all twelve scenarios costs at most 0.83 Wh, and training the four learned arms once costs about 0.22 kWh, both upper bounds from wall-clock time. Direct RAPL/NVML measurement [23, 91] and incremental caching of structural metrics across commits are the main open levers.

8.2. When Graph Learning Helps, and When It Does Not

Practitioners also need to know in which architectures a learned model is worth training, whether a homogeneous or a heterogeneous model suits them, and where learning fails. Table 12 collects the evidence, which is post hoc and exploratory, drawn from the artifacts of Section 7 (`reproduce/engine_regimes.py`; per-fold and per-system detail in Supplementary Section S31).

Where learned engines help. Learned engines are steady across architectures and the closed-form engine is not: by tercile, HGT-QoS scores 0.604, 0.589 and 0.672, whereas Topo-QoS scores 0.324, 0.561 and 0.775. Which engine wins on an architecture is therefore decided by how well centrality happens to fit it. Learned

Table 11: Recommended instruments of the SaG portfolio, given the evidence in Section 7. **Topo-QoS** and the raw-multigraph learned and hybrid engines are omitted, because the dependency counts match or exceed them both on held-out architectures and on the system models.

Instrument	Context	Role and evidence
InDeg (dependency count)	CI gates; architectures resembling the training corpus	Training-free, milliseconds; $\rho = 0.764$ out of distribution, above Topo-QoS on 12/12 folds and above raw connection counts by +0.565.
Reach (dependency count)	CI gates; systems unlike the training corpus, including RPC-derived models	Training-free; $\rho = 0.938$ on the five system models, $\rho_{>0} = 0.871$ on their active components. On both RPC-derived models $\rho_{>0} = 0.813\text{--}0.976$. Every pure learned engine’s $\rho_{>0}$ lies in $[-0.19, +0.16]$ on both; synchronous calls are not modeled (Section 8.4).
GAT-P-QoS (learned, dependency graph)	A learned engine, where richer inputs or oracles are available	$\rho = 0.748$, level with InDeg ; +0.113 over the same engine on the raw multigraph and +0.106 over the feature-only GBM-Feat .
RM explanation layer	Refactoring and root-cause discussion	ISO/IEC 25010 attribution (Availability vs. Fault Tolerance vs. Maintainability).

Table 12: Where graph learning helps. LOSO folds are grouped into terciles of the closed-form engine’s per-fold accuracy (**Topo-QoS** ρ); system models are grouped by the paradigm of the original system. Mean Spearman ρ against $I^*(v)$, Application population; $\rho_{>0}$: active stratum. For system models, “learned” ranges over the five pure learned engines (**GAT**, **GAT-QoS**, **GAT-w**, **HGT-QoS**, **HGT-QoS-U**; **GAT-w** is the scalar-weight capacity control of Supplementary Section S24) and “closed-form” over **Topo** and **Topo-QoS**. The last column gives, on the same folds and systems, the training-free dependency counts (Amendment 7) and the learners on the dependency graph (Amendment 9). Exploratory.

Regime	Where	Best choice	Evidence: raw-multigraph engines	Dependency graph
Closed-form ranks poorly	Microservices, ATM, IoT Smart City, Healthcare	Dependency count or dependency-graph learner	Topo-QoS 0.324; HGT-QoS 0.604, GAT-QoS 0.626, both 4/4 folds. The prior trims the gain (hybrids 0.502 / 0.554).	InDeg 0.693, Reach 0.700, GAT-P-QoS 0.716.
Intermediate	ESB, Telecom RAN, Financial Trading, Industrial SCADA	Dependency count or dependency-graph learner	Topo-QoS 0.561; HGT-QoS 0.589, GAT-QoS 0.660; Hybrid-HGT 0.681, Hybrid-GAT 0.697, both 4/4 folds.	InDeg 0.742, GAT-P-QoS 0.760.
Closed-form ranks well	Logistics Fleet, AV System, Enterprise, Real-Time Gaming	InDeg	Topo-QoS 0.775; HGT-QoS 0.672 (1/4 folds), GAT-QoS 0.620 (0/4); hybrids 0.786 / 0.798.	InDeg 0.858, GAT-P-QoS 0.768.
Unlike the corpus, originally pub-sub	Autoware, EdgeX, Home Assistant	Reach	Learned 0.716–0.927 vs. closed-form 0.289–0.534; learned $\rho_{>0}$ 0.183–0.833 where closed-form is negative.	Reach 0.836–0.997; $\rho_{>0}$ 0.674–0.971.
Unlike the corpus, originally RPC	Online Boutique, Train-Ticket models	Reach	Learned 0.710–0.810, but Topo 0.891 on Online Boutique; learned $\rho_{>0}$ -0.19 to $+0.16$.	Reach 0.966–0.998; $\rho_{>0}$ 0.813–0.976.

engines win every fold where centrality ranks poorly, such as the Microservices and ATM meshes, and on topologies whose symmetric stars create betweenness ties (on EdgeX, closed-form Overlap@K is 0). Among the raw-multigraph engines, they are also the ones that stay reliable on architectures unlike their training corpus. The engines that read the dependency graph are steadier still: **InDeg** scores 0.693, 0.742 and 0.858 across the terciles and **GAT-P-QoS** 0.716, 0.760 and 0.768, and **Reach** ranks every system model at 0.836 or above (Table 12). Because the terciles are defined with the labels, no architecture can be placed in one before deployment, and no descriptor we measured predicts it (see below). Among the raw-multigraph engines, the hybrid is therefore the safe default in distribution. It matches or beats both of its components in two terciles, and it gives up 0.07–0.10 against its learned engine where centrality is weakest.

On the raw multigraph, what succeeds is learning over graph-derived features. Three controls locate the raw-multigraph engines’ accuracy. First, gradient-boosted trees over the same per-node features, with no graph (**GBM-Feat**, an exploratory baseline since Amendment 3), reach $\rho = 0.642$ under LOSO, level with **GAT-QoS** (0.635). Second, every relation on the native substrate points away from Applications, so the

GATs, which pass messages along edge direction, never deliver one to the population they score: deleting every edge leaves their Application outputs unchanged. Third, HGT reaches Applications only through its reverse pass, and removing it costs nothing (HGT-QoS-U, Section 7.2). The learned engines thus succeed as learned combiners of the graph-derived node features of Section 3.4, such as centralities, the Connectivity Degradation Index and QoS coupling weights, rather than through relational reasoning at inference. The neural models add value over trees in transfer: zero-shot, GAT-QoS reaches $\rho = 0.805$ and $\rho_{>0} = 0.319$, against 0.757 and 0.156 for GBM-Feat. In distribution, trees lead on the active stratum (0.391 vs. 0.338). Claims for message passing on similar tasks should be checked against such a feature-only learner. On the dependency graph the picture changes: every component receives messages from its dependents, and GAT-P-QoS beats GBM-Feat by +0.106 (Section 7.2).

Why learning fails where it fails. (i) The target is nearly topological: a topology-only relabeling recovers I^* 's ordering at $\rho = 0.965$ (Section 4.3), and the features already contain the centralities that approximate it. Where centrality ranks well, a learned fit to eleven other architectures replaces a near-sufficient statistic and loses, most on Enterprise (0.426 vs. 0.795). (ii) Training support is narrow: twelve graphs from one generator, and Enterprise, with 300 applications against at most 200 elsewhere, lies outside it. (iii) RPC failure semantics are absent: neither the features nor the forward-propagating oracle encodes synchronous calls, so no learned engine reliably orders the active components of the RPC-derived models, although transitive reach does ($\rho_{>0} \geq 0.813$). (iv) Between 21% and 52% of each population is inert, so much of any full-population ρ separates harmless from harmful components rather than ordering them (Section 7.1). What matters is *which* messages reach a component, not how many: on the raw multigraph the share of the graph from which HGT reaches an Application (35–59% per fold) is uncorrelated with its per-fold ρ (Spearman +0.12) or with its gain over Topo-QoS (−0.06), whereas giving each component the messages of its dependents, on the dependency graph, lifts every GAT arm by +0.075 to +0.113 (Section 7.2).

Homogeneous or heterogeneous? On this evidence no scenario class calls for a heterogeneous model. At matched capacity, typing adds nothing on average (−0.014). HGT-QoS wins 4 of 12 folds against GAT-QoS (ATM, Enterprise, Logistics Fleet, Real-Time Gaming), and these share no descriptor we measured. The untyped engine also transfers better on all five system models. One exploratory pattern deserves a test. Without QoS inputs, HGT's margin over GAT grows with scale: across folds, its Spearman correlation is +0.63 with the number of applications, +0.66 with topics and +0.77 with brokers (Enterprise +0.122, Healthcare −0.131). The pattern vanishes once QoS inputs are present (+0.03 with applications). Typed parameters may therefore stand in for undeclared QoS contracts in large multi-broker systems. However, this is one of 96 descriptor–gain correlations, none of which survives a Benjamini–Hochberg correction ($q \geq 0.32$), so it is a hypothesis rather than a finding. The untyped engine is the default we recommend. On scale more generally, across folds of 26 to 300 applications, the learned engines' gain over Topo-QoS falls with size (Spearman −0.48 with applications, −0.62 with libraries), while the hybrid's correction gains value (+0.43), again without surviving correction. Ranking accuracy was not measured above 300 applications; the 2,000-component rows of Table 10 measure cost only.

8.3. Threats to Validity

Construct validity. All labels are simulator-derived rather than observed failures. The behavioral queue-flow oracle agrees with the primary cascade oracle at $\rho = 0.627$ (Section 4.3); much of that agreement concerns which components are harmless. Because $I^*(v)$ is a reachability functional of the topology the predictors read, a strong closed-form comparator is expected, and this bears on how the dependency counts should be read. InDeg is publish–subscribe afferent coupling [52, 53] in the graph along which the oracle's cascade travels, so its accuracy partly measures how consistent SaG's dependency rules are with the oracle, not agreement with observed failures. What the evidence does establish is that the derivation is necessary for the count to work: raw connection counts on the same components score 0.199 (Amendment 10), and the derived library rule adds to transitive reach. Retargeting the LOSO contrasts on I_{dyn} is the next experiment. The five system models are the most judgement-laden input: each was written by one author, no second modeler has re-derived them, and RQ3's figures hold only for these models. The replication package includes a re-modeling protocol and an agreement tool (`reproduce/model_agreement.py`) so that this check is cheap to run.

Internal validity. Predictors consume G_{analysis} and oracles traverse $G_{\text{structural}}$, which is enforced in CI. Substrate, training set, depth and early stopping are matched across learned arms. All typing conclusions rest on the capacity- and channel-matched control, and all four registered control arms have run, so capacity, edge-channel width and message directionality are each matched (Section 8.2). The dependency-graph learners keep their raw-multigraph counterparts’ parameter budgets and bit-identical node features and labels, so only the graph they read differs. The regime analysis of Section 8.2 is post hoc. No hyperparameter was tuned on an evaluation split.

Robustness to free parameters. Under Morris screening only two of ten declared constants, the AHP shrinkage λ and r_{FT} , have appreciable influence ($\mu^* \approx 0.13$ against ≤ 0.025 for the rest), and no setting of the topic-weight or QoS sub-weight constants changes any reported comparison (Supplementary Sections S1–S4).

External validity. The synthetic corpus comes from one generator family, and the system models are small (22–41 applications). Scaling beyond 2,000 nodes would benefit from incremental caching or mini-batching [92].

Conclusion validity and repeatability. LOSO folds share training scenarios, so p -values are nominal [90] and are read alongside fold-level sign consistency and bootstrap intervals. At fixed code, seeds and device, every figure reproduces at its reported precision, and all training-free cells also reproduce across devices. Learned cells move across code revisions and devices (up to 0.172 in a fold mean; HGT-QoS 0.041), so learned figures are reported against the released artifacts, and every comparison is made within one sweep. The drift ledger is in the replication repository (`reproduce/rerun_drift.py`).

8.4. Limitations and Future Work

The explanation layer’s attributions have not been evaluated with developers or against injected faults, and no published learned-criticality model (FINDER [67], DrBC [68]) has been reproduced on this corpus. Next steps are: (1) an independent re-model of at least two of the five systems, with inter-modeler agreement reported; (2) learned corrections that exceed the dependency counts rather than match them, which oracles with QoS semantics would make possible; (3) retargeting RQ1 and RQ2, including the dependency counts, on I_{dyn} and on the composite oracle of Supplementary Section S1.2, and validating Rules 2–4 and 6, which this evaluation of the Application population does not validate; (4) synchronous call edges and a backward-propagating oracle, so that RPC architectures can be modeled natively; (5) extracting system models from deployment manifests; and (6) validating rankings against production incident data.

9. Conclusion

Deriving explicit dependencies from architecture models is what makes cascading-failure risk in publish–subscribe systems rankable before deployment. SaG turns an architecture into a typed multigraph and, through six publish–subscribe rules, into a graph of logical dependencies that exposes the paths along which failures travel. On twelve held-out synthetic architectures, counting a component’s dependents on that graph ranks its simulated cascade impact at $\rho = 0.764$, above closed-form centrality on every fold, while counting its raw connections reaches only 0.199. On independently authored models of five open-source systems, counting transitive dependents reaches $\rho = 0.938$ with no training at all. The same graph lifts every other analyzer: centrality on the Application–Library dependency graph gains +0.204 on every fold, and graph neural networks gain +0.075 to +0.113 when they read it instead of the raw multigraph, reaching $\rho = 0.748$, level with the dependency counts. Hybrid engines, in which a learned model corrects closed-form centrality, beat it on 11 of 12 folds under decision rules registered before their runs, and remain significant under a Holm correction over all thirteen confirmatory contrasts. Learned engines need messages from a component’s dependents and declared coupling inputs, not relation-specific parameters, so a simple untyped attention network suffices, and trained only on synthetic data they transfer far above centrality (0.760–0.831 against 0.511–0.526).

For practice, the most valuable step is to model the architecture with typed entities and derive its dependencies. A CI gate can then count dependents in milliseconds, `InDeg` for familiar architectures and `Reach` for unfamiliar ones, and a learned engine that reads the dependency graph can be added where richer

inputs are available. SaG’s ISO/IEC 25010 layer names the remediation each flagged component calls for. Because the synthetic corpus regenerates byte-identically, every registered decision rule is applied as written, and every reported value is reconciled against released artifacts, others can extend or challenge these results directly.

Three steps would widen their reach: retargeting the counts and the engines on the behavioral queue-flow oracle, and on oracles that express QoS semantics such as deadline misses, where learned corrections could exceed the counts; independent re-models of the five systems; and extracting models from deployment manifests, so that rankings can be validated against production incidents.

Declarations

CRedit Authorship Contribution Statement. **Ibrahim Onuralp Yigit:** Conceptualization, Methodology, Software, Validation, Formal analysis, Investigation, Data curation, Writing – original draft, Visualization. **Feza Buzluca:** Conceptualization, Methodology, Validation, Writing – review and editing, Supervision. **Declaration of Competing Interest.** The authors declare no competing financial interests or personal relationships that could have influenced this work. **Funding.** This research received no external grant. **Data Availability.** The replication package (datasets, harnesses, checkpoints, scripts) is available on Zenodo under DOI 10.5281/zenodo.14922108 [93] with `uv/pip` environments. The public repository documents every experiment reported here — protocol, hyperparameters, reproduction command, artifacts and extended results — at <https://github.com/onuralpyigit/software-as-a-graph/tree/jss-submission-v4/docs/research/jss/experiments>. Synthetic datasets regenerate byte-identically. The deposit ships all artifacts backing reported tables as a dated bundle (`SaG_JSS_Results_<stamp>`) with a `MANIFEST.json` recording SHA-256 digests, commit hashes, and corpus provenance. Four supplementary artifacts predate provenance stamping and carry no commit or corpus digest: `atm_scale_sweep_v3.json` (S6), `qos_label_ablation.json` (Section 4.3), `threshold_sensitivity_v3.json` (S3) and `topic_weight_sensitivity_v3.json` (S1); their correspondence to the corpus is asserted by the bundle rather than recorded in the file. The verification script (`reproduce/reconcile_manuscript.py`) runs standalone against the deposit, mechanically verifying 511 reported figures in the manuscript and supplement against the JSON artifacts.

Declaration of Generative AI and AI-assisted technologies in the manuscript preparation process

During the preparation of this work the authors used Anthropic’s Claude to assist with typesetting, LaTeX formatting, and the development of analysis and reporting scripts in the replication package. After using this tool the authors reviewed and edited the content as needed and take full responsibility for the content of the published article. The study design, the choice of experiments, the interpretation of results, and all scientific claims are the authors’ own.

References

- [1] S. Macenski, T. Foote, B. Gerkey, C. Lalancette, W. Woodall, Robot operating system 2: Design, architecture, and uses in the wild, *Science Robotics* 7 (66) (2022) eabm6074.
- [2] J. Kreps, N. Narkhede, J. Rao, Kafka: A distributed messaging system for log processing, in: *Proc. 6th Int. Workshop on Networking Meets Databases (NetDB)*, 2011.
- [3] Object Management Group, Data distribution service (dds), Tech. Rep. formal/2015-04-10, version 1.4, Object Management Group (2015).
- [4] OASIS, MQTT version 5.0, OASIS Standard, <https://docs.oasis-open.org/mqtt/mqtt/v5.0/mqtt-v5.0.html> (accessed 9 September 2026) (2019).
- [5] N. Dragoni, S. Giallorenzo, A. L. Lafuente, M. Mazzara, F. Montesi, R. Mustafin, L. Safina, *Microservices: Yesterday, today, and tomorrow*, in: *Present and Ulterior Software Engineering*, Springer, 2017, pp. 195–216.
- [6] S. Newman, *Building Microservices: Designing Fine-Grained Systems*, O’Reilly Media, 2015.
- [7] P. T. Eugster, P. A. Felber, R. Guerraoui, A.-M. Kermarrec, The many faces of publish/subscribe, *ACM Computing Surveys* 35 (2) (2003) 114–131.
- [8] A. E. Motter, Y.-C. Lai, Cascade-based attacks on complex networks, *Physical Review E* 66 (2002) 065102(R).
- [9] S. V. Buldyrev, R. Parshani, G. Paul, H. E. Stanley, S. Havlin, Catastrophic cascade of failures in interdependent networks, *Nature* 464 (2010) 1025–1028.
- [10] R. Albert, H. Jeong, A.-L. Barabási, Error and attack tolerance of complex networks, *Nature* 406 (2000) 378–382.

- [11] A. Avizienis, J.-C. Laprie, B. Randell, C. Landwehr, Basic concepts and taxonomy of dependable and secure computing, *IEEE Transactions on Dependable and Secure Computing* 1 (1) (2004) 11–33.
- [12] L. Bass, P. Clements, R. Kazman, *Software Architecture in Practice*, 3rd Edition, Addison-Wesley, 2012.
- [13] D. E. Perry, A. L. Wolf, Foundations for the study of software architecture, *ACM SIGSOFT Software Engineering Notes* 17 (4) (1992) 40–52.
- [14] J. Garcia, D. Popescu, G. Edwards, N. Medvidovic, Identifying architectural bad smells, in: *Proc. 13th European Conf. on Software Maintenance and Reengineering (CSMR)*, 2009, pp. 255–258.
- [15] R. Kazman, M. Klein, M. Barbacci, T. Longstaff, H. Lipson, J. Carriere, The architecture tradeoff analysis method, in: *Proc. 4th IEEE Int. Conf. on Engineering of Complex Computer Systems (ICECCS)*, 1998, pp. 68–78.
- [16] SonarSource, Clean as you code, SonarQube documentation, <https://docs.sonarsource.com/sonarqube-server/latest/core-concepts/clean-as-you-code/introduction/> (accessed 9 September 2026) (2024).
- [17] S. R. Chidamber, C. F. Kemerer, A metrics suite for object oriented design, *IEEE Transactions on Software Engineering* 20 (6) (1994) 476–493.
- [18] A. Basiri, N. Behnam, R. de Rooij, L. Hochstein, L. Kosewski, J. Reynolds, C. Rosenthal, Chaos engineering, *IEEE Software* 33 (3) (2016) 35–41.
- [19] L. C. Freeman, A set of measures of centrality based on betweenness, *Sociometry* 40 (1) (1977) 35–41.
- [20] U. Brandes, A faster algorithm for betweenness centrality, *Journal of Mathematical Sociology* 25 (2) (2001) 163–177.
- [21] I. O. Yigit, F. Buzluca, A graph-based dependency analysis method for identifying critical components in distributed publish-subscribe systems, in: *Proc. IEEE Int. Conf. on Recent Advances in Systems Science and Engineering (RASSE)*, 2025, pp. 1–8. doi:10.1109/RASSE64831.2025.11315354.
- [22] C. Calero, M. Piattini (Eds.), *Green in Software Engineering*, Springer, Cham, Switzerland, 2015. doi:10.1007/978-3-319-08581-4.
- [23] L. Lannelongue, J. Grealey, M. Inouye, Green algorithms: Quantifying the carbon footprint of computation, *Advanced Science* 8 (12) (2021) 2100707. doi:10.1002/adv.202100707.
- [24] R. Verdecchia, J. Sallou, L. Cruz, A systematic review of Green AI, *WIREs Data Mining and Knowledge Discovery* 13 (4) (2023) e1507. doi:10.1002/widm.1507.
- [25] R. C. Cheung, A user-oriented software reliability model, *IEEE Transactions on Software Engineering SE-6* (2) (1980) 118–125.
- [26] K. Goseva-Popstojanova, K. S. Trivedi, Architecture-based approach to reliability assessment of software systems, *Performance Evaluation* 45 (2–3) (2001) 179–204.
- [27] A. Immonen, E. Niemelä, Survey of reliability and availability prediction methods from the architectural perspective, *Software and Systems Modeling* 7 (1) (2008) 49–65.
- [28] S. Becker, H. Koziol, R. Reussner, The Palladio component model for model-driven performance prediction, *Journal of Systems and Software* 82 (1) (2009) 3–22.
- [29] G. Franks, T. Al-Omari, M. Woodside, O. Das, S. Derisavi, Enhanced modeling and solution of layered queueing networks, *IEEE Transactions on Software Engineering* 35 (2) (2009) 148–161.
- [30] J. Delange, P. H. Feiler, Architecture fault modeling with the AADL error-model annex, in: *2014 40th EUROMICRO Conference on Software Engineering and Advanced Applications (SEAA)*, IEEE, 2014, pp. 361–368. doi:10.1109/SEAA.2014.20.
- [31] Y. Gan, Y. Zhang, K. Hu, D. Cheng, Y. He, M. Pancholi, C. Delimitrou, Seer: Leveraging big data to navigate the complexity of performance debugging in cloud microservices, in: *Proc. ACM Int. Conf. on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2019.
- [32] Y. Gan, M. Liang, S. Dev, D. Lo, C. Delimitrou, Sage: Practical and scalable ML-driven performance debugging in microservices, in: *Proc. ACM Int. Conf. on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2021.
- [33] L. Wu, J. Tordsson, E. Elmroth, O. Kao, MicroRCA: Root cause localization of performance issues in microservices, in: *Proc. IEEE/IFIP Network Operations and Management Symposium (NOMS)*, 2020.
- [34] Z. Li, J. Chen, R. Jiao, N. Zhao, Z. Wang, S. Zhang, Y. Wu, L. Jiang, L. Yan, Z. Wang, Z. Chen, W. Zhang, X. Nie, K. Sui, D. Pei, Practical root cause localization for microservice systems via trace analysis, in: *Proc. IEEE/ACM Int. Symposium on Quality of Service (IWQoS)*, 2021.
- [35] C. Zhang, X. Peng, C. Sha, K. Zhang, Z. Fu, X. Wu, Q. Lin, D. Zhang, DeepTraLog: Trace-log combined microservice anomaly detection through graph-based deep learning, in: *Proc. IEEE/ACM Int. Conf. on Software Engineering (ICSE)*, 2022.
- [36] C. Lee, T. Yang, Z. Chen, Y. Su, Y. Yang, M. R. Lyu, Eadro: An end-to-end troubleshooting framework for microservices on multi-source data, in: *Proc. IEEE/ACM Int. Conf. on Software Engineering (ICSE)*, 2023.
- [37] X. Meng, P. Shen, Y. Sun, D. Liu, J. Lu, S. Zhang, D. Pei, Microcause: Root cause analysis for microservice systems through graph neural networks, in: *Proc. IEEE International Conference on Software Maintenance and Evolution (ICSME)*, 2020, pp. 403–414.
- [38] S. Zhang, S. Xia, W. Fan, B. Shi, X. Xiong, Z. Zhong, M. Ma, Y. Sun, D. Pei, Failure diagnosis in microservice systems: A comprehensive survey and analysis, *ACM Transactions on Software Engineering and Methodology* (2025). doi:10.1145/3715005.
- [39] X. Zhou, X. Peng, T. Xie, J. Sun, C. Ji, W. Li, D. Ding, Fault analysis and debugging of microservice systems: Industrial survey, benchmark system, and empirical study, *IEEE Transactions on Software Engineering* 47 (2) (2021) 243–260.
- [40] S. A. Bohner, R. S. Arnold, *Software Change Impact Analysis*, IEEE Computer Society Press, Los Alamitos, CA, 1996.
- [41] S. Esparrachieri, T. Reilly, A. Rentz, Tracking and controlling microservice dependencies, *ACM Queue* 16 (4) (2018). doi:10.1145/3277539.3277541.
- [42] X. Yang, K. Tang, X. Yao, A learning-to-rank approach to software defect prediction, *IEEE Transactions on Reliability* 64 (1) (2015) 234–246.
- [43] T. J. McCabe, A complexity measure, *IEEE Transactions on Software Engineering SE-2* (4) (1976) 308–320.
- [44] N. Fenton, J. Bieman, *Software Metrics: A Rigorous and Practical Approach*, 3rd Edition, CRC Press, 2014.
- [45] V. R. Basili, L. C. Briand, W. L. Melo, A validation of object-oriented design metrics as quality indicators, *IEEE Transactions on Software Engineering* 22 (10) (1996) 751–761.
- [46] N. Nagappan, T. Ball, Static analysis tools as early indicators of pre-release defect density, in: *Proc. 27th Int. Conf. on Software Engineering (ICSE)*, 2005, pp. 580–586.
- [47] T. Zimmermann, R. Premraj, A. Zeller, Predicting defects for Eclipse, in: *Proc. 3rd Int. Workshop on Predictor Models in Software Engineering (PROMISE)*, 2007.

- [48] T. Menzies, J. Greenwald, A. Frank, Data mining static code attributes to learn defect predictors, *IEEE Transactions on Software Engineering* 33 (1) (2007) 2–13.
- [49] V. Bushong, D. Das, A. Al Maruf, T. Cerny, Using static analysis to address microservice architecture reconstruction, in: 2021 36th IEEE/ACM International Conference on Automated Software Engineering (ASE), IEEE, 2021. doi: 10.1109/ASE51524.2021.9678749.
- [50] S. Schneider, A. Bakhtin, X. Li, J. Soldani, A. Brogi, T. Cerny, R. Scandariato, D. Taibi, Comparison of static analysis architecture recovery tools for microservice applications, *arXiv preprint* (2024). [arXiv:2412.08352](#), doi:10.48550/arXiv.2412.08352.
- [51] A. Santos, A. Cunha, N. Macedo, Statistical and model-driven static analysis of ROS systems, *IEEE Transactions on Software Engineering* 47 (10) (2019) 2200–2218.
- [52] R. C. Martin, *Agile Software Development: Principles, Patterns, and Practices*, Prentice Hall, 2003.
- [53] D. Rud, A. Schmietendorf, R. R. Dumke, Product metrics for service-oriented infrastructures, in: *Applied Software Measurement: Proceedings of the International Workshop on Software Metrics and DASMA Software Metrik Kongress (IWSM/MetriKon 2006)*, Shaker Verlag, 2006.
- [54] J. Bogner, S. Wagner, A. Zimmermann, Automatically measuring the maintainability of service- and microservice-based systems: A literature review, in: *Proceedings of the 27th International Workshop on Software Measurement and 12th International Conference on Software Process and Product Measurement (IWSM Mensura '17)*, ACM, 2017, pp. 107–115.
- [55] J. Garcia, D. Popescu, G. Edwards, N. Medvidovic, Toward a catalogue of architectural bad smells, in: *Proc. 5th Int. Conf. on the Quality of Software Architectures (QoSA)*, LNCS 5581, 2009, pp. 146–162.
- [56] D. Taibi, V. Lenarduzzi, On the definition of microservice bad smells, *IEEE Software* 35 (3) (2018) 56–62.
- [57] Z. Li, P. Avgeriou, P. Liang, A systematic mapping study on technical debt and its management, *Journal of Systems and Software* 101 (2015) 193–220.
- [58] J. Humble, D. Farley, *Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation*, Addison-Wesley, 2010.
- [59] L. Chen, Continuous delivery: Huge benefits, but challenges too, *IEEE Software* 32 (2) (2015) 50–54.
- [60] International Organization for Standardization, ISO/IEC 25010:2023 — systems and software engineering — systems and software quality requirements and evaluation (square) — product quality model, Tech. rep., International Organization for Standardization (2023).
- [61] International Organization for Standardization, ISO/IEC 25019:2023 — systems and software engineering — systems and software quality requirements and evaluation (square) — quality-in-use model, Tech. rep., International Organization for Standardization (2023).
- [62] International Organization for Standardization, ISO/IEC 25023:2016 — systems and software engineering — systems and software quality requirements and evaluation (square) — measurement of system and software product quality, Tech. rep., International Organization for Standardization (2016).
- [63] International Organization for Standardization, ISO/IEC 25021:2012 — systems and software engineering — systems and software quality requirements and evaluation (square) — quality measure elements, Tech. rep., International Organization for Standardization (2012).
- [64] T. L. Saaty, *The Analytic Hierarchy Process: Planning, Priority Setting, Resource Allocation*, McGraw-Hill, 1980.
- [65] S. Brin, L. Page, The anatomy of a large-scale hypertextual web search engine, *Computer Networks and ISDN Systems* 30 (1–7) (1998) 107–117.
- [66] M. E. J. Newman, *Networks: An Introduction*, Oxford University Press, 2010.
- [67] C. Fan, L. Zeng, Y. Sun, Y.-Y. Liu, Finding key players in complex networks through deep reinforcement learning, *Nature Machine Intelligence* 2 (2020) 317–324.
- [68] C. Fan, L. Zeng, Y. Ding, M. Chen, Y. Sun, Z. Liu, Learning to identify high betweenness centrality nodes from scratch: A novel graph neural network approach, in: *Proc. 28th ACM Int. Conf. on Information and Knowledge Management (CIKM)*, 2019, pp. 559–568.
- [69] A. Varbella, K. Amara, M. El-Assady, B. Gjorgiev, G. Sansavini, PowerGraph: A power grid benchmark dataset for graph neural networks, in: *Advances in Neural Information Processing Systems 37 (NeurIPS 2024)*, Datasets and Benchmarks Track, 2024, [arXiv:2402.02827](#).
- [70] T. N. Kipf, M. Welling, Semi-supervised classification with graph convolutional networks, in: *Proc. Int. Conf. on Learning Representations (ICLR)*, 2017.
- [71] W. L. Hamilton, R. Ying, J. Leskovec, Inductive representation learning on large graphs, in: *Advances in Neural Information Processing Systems 30 (NeurIPS)*, 2017, pp. 1024–1034.
- [72] P. Velićković, G. Cucurull, A. Casanova, A. Romero, P. Liò, Y. Bengio, Graph attention networks, in: *Proc. Int. Conf. on Learning Representations (ICLR)*, 2018.
- [73] M. Schlichtkrull, T. N. Kipf, P. Bloem, R. van den Berg, I. Titov, M. Welling, Modeling relational data with graph convolutional networks, in: *Proc. European Semantic Web Conference (ESWC)*, 2018, pp. 593–607.
- [74] X. Wang, H. Ji, C. Shi, B. Wang, Y. Ye, P. Cui, P. S. Yu, Heterogeneous graph attention network, in: *Proc. The Web Conference (WWW)*, 2019, pp. 2022–2032.
- [75] Z. Hu, Y. Dong, K. Wang, Y. Sun, Heterogeneous graph transformer, in: *Proc. The Web Conference (WWW)*, 2020, pp. 2704–2710.
- [76] X. Fu, J. Zhang, Z. Meng, I. King, MAGNN: Metapath aggregated graph neural network for heterogeneous graph embedding, in: *Proc. The Web Conference (WWW)*, 2020, pp. 2331–2341.
- [77] G. Khodabandeh, A. Ezaz, M. Babaei, N. Ezzati-Jivan, Utilizing graph neural networks for effective link prediction in microservice architectures, in: *Proceedings of the 16th ACM/SPEC International Conference on Performance Engineering (ICPE)*, 2025.
- [78] Z. Ying, D. Bourgeois, J. You, M. Zitnik, J. Leskovec, GNNExplainer: Generating explanations for graph neural networks, in: *Advances in Neural Information Processing Systems (NeurIPS)*, Vol. 32, 2019, pp. 9244–9255.
- [79] D. Luo, W. Cheng, D. Xu, W. Yu, B. Zong, H. Chen, X. Zhang, Parameterized explainer for graph neural network, in: *Advances in Neural Information Processing Systems (NeurIPS)*, Vol. 33, 2020, pp. 19620–19631.
- [80] J. Pearl, *Probabilistic Reasoning in Intelligent Systems: Networks of Plausible Inference*, Morgan Kaufmann, 1988.
- [81] G. Beliakov, A. Pradera, T. Calvo, *Aggregation functions: A guide for practitioners*, Studies in Fuzziness and Soft Computing 221 (2007).
- [82] R. R. Yager, On ordered weighted averaging aggregation operators in multicriteria decisionmaking, *IEEE Transactions on Systems, Man, and Cybernetics* 18 (1) (1988) 183–190.
- [83] G. H. Hardy, J. E. Littlewood, G. Pólya, *Inequalities*, 2nd Edition, Cambridge University Press, 1952.

- [84] M. Fey, J. E. Lenssen, Fast graph representation learning with PyTorch geometric, in: ICLR Workshop on Representation Learning on Graphs and Manifolds, 2019.
- [85] F. Xia, T.-Y. Liu, J. Wang, W.-S. Zhang, H. Li, Listwise approach to learning to rank: Theory and algorithm, in: Proc. 25th Int. Conf. on Machine Learning (ICML), 2008, pp. 1192–1199.
- [86] Team SimPy, Simpy: Discrete event simulation for Python, Software, <https://simpy.readthedocs.io> (accessed 9 September 2026) (2020).
- [87] F. Wilcoxon, Individual comparisons by ranking methods, Biometrics Bulletin 1 (6) (1945) 80–83.
- [88] B. Efron, R. J. Tibshirani, An Introduction to the Bootstrap, Chapman & Hall, 1993.
- [89] C. Spearman, The proof and measurement of association between two things, American Journal of Psychology 15 (1) (1904) 72–101.
- [90] C. Nadeau, Y. Bengio, Inference for the generalization error, Machine Learning 52 (2003) 239–281. doi:10.1023/A:1024068626366.
- [91] V. Schmidt, K. Goyal, A. Joshi, B. Feld, L. Conell, N. Laskaris, D. Sarthou, H. Verreault, J. Blank, S. Zhang, Codecarbon: Estimate and track carbon emissions from machine learning computing, Journal of Open Source Software (2021).
- [92] H. Zeng, H. Zhou, A. Srivastava, R. Kannan, V. Prasanna, GraphSAINT: Graph sampling based inductive engine, in: Proc. International Conference on Learning Representations (ICLR), 2020.
- [93] I. O. Yigit, F. Buzluca, [dataset] software-as-a-graph: Replication package (datasets, generator configurations, simulation harnesses, model checkpoints, and analysis scripts), <https://doi.org/10.5281/zenodo.14922108> (2026). doi:10.5281/zenodo.14922108.